



redis

Redis NoSQL — Mise en œuvre

Formateur : Anas Ben Yaiche

API · CLUSTERING · SURVEILLANCE · OPTIMISATION DU MODÈLE DE DONNÉES

Objectifs pédagogiques

 The picture can't be displayed.

Maîtriser les cas d'usage Redis

- 1** Mettre en œuvre Redis à travers ses principaux scénarios : cache, sessions, pub/sub, files de messages, géolocalisation et séries temporelles.

 The picture can't be displayed.

Utiliser Redis comme boîte à outils

- 2** Identifier la structure de données adaptée à chaque problème et exploiter l'API Redis de manière fluide et idiomatique depuis différents clients.

 The picture can't be displayed.

Optimiser Redis en production

- 3** Connaître les techniques d'optimisation des performances, du modèle de données, de la haute disponibilité, du monitoring et de la sécurité.

Public concerné & Prérequis



Développeurs

Intégration de Redis dans des applications web ou microservices.



Architectes

Conception de systèmes distribués et choix de la topologie Redis.



Ingénieurs de la donnée

Traitement de flux de données avec Streams et structures avancées.

Prérequis



Bases de programmation

Maîtrise d'au moins un langage (Java, Node.js, Python...) et notions de structures de données.



Aucune connaissance préalable de Redis n'est requise.

Programme de la formation

1 Module 1 — Introduction

Histoire et positionnement de Redis. Modèle clé-valeur en mémoire, persistance RDB/AOF, premiers pas avec `redis-cli`.

2 Module 2 — Structures de données

Strings, Lists, Sets, Sorted Sets, Hashes, Bitmaps, HyperLogLog. Commandes fondamentales, TTL et politiques d'expiration.

3 Module 3 — Architecture

Modèle mono-thread, event loop, persistance, réplication maître-réplique, Sentinel pour la haute disponibilité.

4 Module 4 — Développer avec Redis

Intégration applicative avec Jedis, Spring Data Redis (Java) et node-redis (Node.js). Pipelines, transactions MULTI/EXEC et scripting Lua.

5 Module 5 — Haute disponibilité & Clustering

Redis Cluster : sharding, slots, failover automatique. Topologies de déploiement, gestion des nœuds et scénarios de reprise.

6 Module 6 — Optimisation & Structures avancées

Streams pour l'événement sourcing, Geo pour la géolocalisation, Bloom filters. Optimisation mémoire : encodings compacts, `OBJECT ENCODING`.

7 Module 7 — Monitoring

Commande `INFO`, métriques clés, RedisInsight, intégration Prometheus/Grafana. Détection de hot keys et analyse de `SLOWLOG`.

8 Module 8 — Sécurité & Écosystème

ACL, TLS, authentification. Patterns de production : cache-aside, write-through, rate limiting. Panorama de l'écosystème Redis Stack.

Environnement technique du lab

Tous les travaux pratiques s'exécutent dans un environnement conteneurisé, reproductible et prêt à l'emploi.

 The picture can't be displayed.



Redis 7 / 8

Dernières versions stables de Redis, avec les nouvelles fonctionnalités (LMPOP, SINTERCARD, Sharded Pub/Sub).

 The picture can't be displayed.



Docker & Compose

Orchestration locale de l'ensemble des services : nœuds Redis, Sentinel, Cluster, et outils de monitoring.

 The picture can't be displayed.



redis-cli

Interface en ligne de commande officielle pour l'exploration interactive, le diagnostic et l'administration.

 The picture can't be displayed.



RedisInsight

Interface graphique officielle Redis pour l'exploration des données, le profiling et le monitoring en temps réel.

 The picture can't be displayed.



Java — Jedis & Spring Data

Clients Java de référence : Jedis pour un accès direct et Spring Data Redis pour l'intégration dans l'écosystème Spring Boot.

 The picture can't be displayed.



Node.js — node-redis

Client officiel Node.js, async/await natif, support des pipelines et des commandes avancées de Redis 7+.

Conseils pour tirer le meilleur parti de la formation

Quelques habitudes simples font toute la différence entre une formation subie et une formation transformatrice.

Pratiquez en dehors des TP

Ouvrez un `redis-cli` pendant les pauses et expérimentez librement : Redis répond en millisecondes, le feedback est immédiat. La curiosité est votre meilleur outil d'apprentissage.

Connectez le contenu à votre contexte

À chaque nouveau pattern ou structure de données, demandez-vous : *où pourrais-je l'utiliser dans mon projet actuel ?* Cette projection mentale ancre durablement les apprentissages.

Posez des questions sans attendre

Le formateur est là pour vous. Une incompréhension non verbalisée en début de journée peut bloquer toute la progression du module suivant. Il n'y a pas de mauvaise question.

Relisez le support après la formation

Le support professionnel est conçu pour être autonome. Une relecture 48 h après la formation consolide les acquis et permet de retrouver rapidement une commande ou un pattern en situation réelle.

📌 Objectif final : repartir avec la confiance de mettre Redis en production dès le lendemain de la formation.

Fondamentaux in-memory

Installation &
redis-cli

Cas d'usage &
positionnement

Écosystème & licences

Module 1 : Introduction à Redis

Comprendre Redis en profondeur : architecture, cas d'usage, écosystème et prise en main — un guide complet pour développeurs et ingénieurs de la donnée.

FORMATION REDIS

MODULE 1 / 1

Qu'est-ce que Redis ?

REmote DIctionary Server — créé en 2009 par Salvatore Sanfilippo, Redis est un serveur de structures de données **in-memory** devenu l'un des outils les plus utilisés dans le monde du développement logiciel.

⚡ Données en RAM

Latences de l'ordre de $\sim 100 \mu s$ Pas d'accès disque : les données vivent entièrement en mémoire vive.

🗄 Structures typées

Strings, Lists, Hashes, Sets, Sorted Sets... manipulées côté serveur, à la différence de Memcached.

🔒 Mono-thread

Atomicité native : chaque commande s'exécute sans interruption. Attention : une commande lente bloque tout.

Redis comme cache en mémoire

Redis s'insère entre l'application et la base de données pour absorber la charge et réduire la latence. Selon le résultat de la recherche en cache, deux chemins sont possibles.



Application

Émet une requête de lecture ou d'écriture

Redis

Cache HIT → réponse immédiate
Cache MISS → délégation à la BDD

Base de données

Données persistantes (PostgreSQL, MySQL...)

Cache-Aside

L'app gère elle-même la lecture du cache

Read-Through

Le cache charge automatiquement depuis la BDD

Write-Through

Écriture simultanée cache + BDD

Write-Behind

Écriture cache immédiate, BDD en asynchrone

Les trois rôles de Redis



Cache haute performance

Réduction drastique de la latence pour les lectures répétitives. Redis sert de couche d'accélération devant n'importe quelle base de données.



Base de données primaire

Gestion des **sessions utilisateur**, des **compteurs temps réel** et des **classements** (leaderboards). Redis devient la source de vérité pour des données à accès fréquent.



Broker de messages

Pub/Sub, Lists comme files d'attente, et **Streams** pour l'ingestion d'événements à haute fréquence — un concurrent léger à Kafka pour certains usages.

Redis et le théorème CAP

Le théorème CAP

Un système distribué ne peut garantir simultanément que deux des trois propriétés suivantes :

- **Consistency** — toutes les lectures voient la dernière écriture
- **Availability** — chaque requête reçoit une réponse
- **Partition tolerance** — le système fonctionne malgré une coupure réseau

Redis se positionne comme un système **CP** (en cluster) : il privilégie la cohérence et la tolérance aux partitions.

Familles NoSQL

- **Clé-valeur** : Redis, Memcached, DynamoDB
- **Document** : MongoDB, CouchDB
- **Colonne** : Cassandra, HBase
- **Graphe** : Neo4j, Amazon Neptune

Redis appartient à la famille clé-valeur mais se distingue par la richesse de ses structures de données et son support natif des Streams.

Redis vs. les autres solutions

Redis n'est pas un remplaçant de votre base relationnelle — c'est un **complémentaire puissant** pour les données dont on connaît la clé d'accès.

Critère	Redis	PostgreSQL	MongoDB	Memcached
Stockage	RAM (+ disque optionnel)	Disque	Disque	RAM uniquement
Modèle	Structures typées	Relationnel	Document JSON	Clé-valeur simple
Requêtes ad hoc	Limitées (Redisearch)	SQL complet	MQL flexible	Aucune
Transactions	MULTI/EXEC (partiel)	ACID complet	Multi-doc depuis 4.0	Aucune
Latence typique	~ 100 µs	1–10 ms	1–5 ms	~ 100 µs



Redis complète la base relationnelle — il ne la remplace pas. On y stocke les données dont on connaît la clé d'accès à l'avance.

Quand utiliser Redis ?

Cas d'usage idéaux

- **Cache applicatif**

Accélération des lectures répétitives

- **Sessions & authentication**

Stockage JWT, cookies, tokens OAuth

- **Files de tâches**

Queues légères avec Lists ou Streams

- **Leaderboards & compteurs**

Sorted Sets pour les classements temps réel

- **Rate limiting**

Limitation de débit par clé (IP, user)

- **Pub/Sub & géolocalisation**

Notifications push, proximité géographique

Quand l'éviter

- Volume de données supérieur à la RAM disponible
- Requêtes relationnelles complexes avec jointures
- Besoin d'un **ACID complet** multi-tables
- Données nécessitant une durabilité absolue sans configuration spécifique
- Analytique lourde ou agrégations sur grands volumes

Redis dans le monde réel

Les plus grandes plateformes mondiales s'appuient sur Redis pour absorber des milliards de requêtes quotidiennes avec une latence minimale.



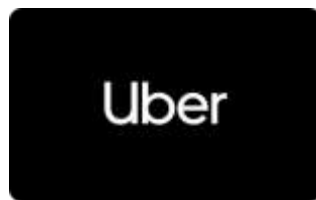
Twitter / X

Cache des timelines et compteurs de métriques sociales en temps réel pour des centaines de millions d'utilisateurs.



GitHub

Gestion des sessions, rate limiting de l'API et file d'attente des jobs CI/CD asynchrones.



Uber

Géolocalisation des chauffeurs en temps réel grâce aux structures GEO de Redis pour des millions de requêtes par seconde.

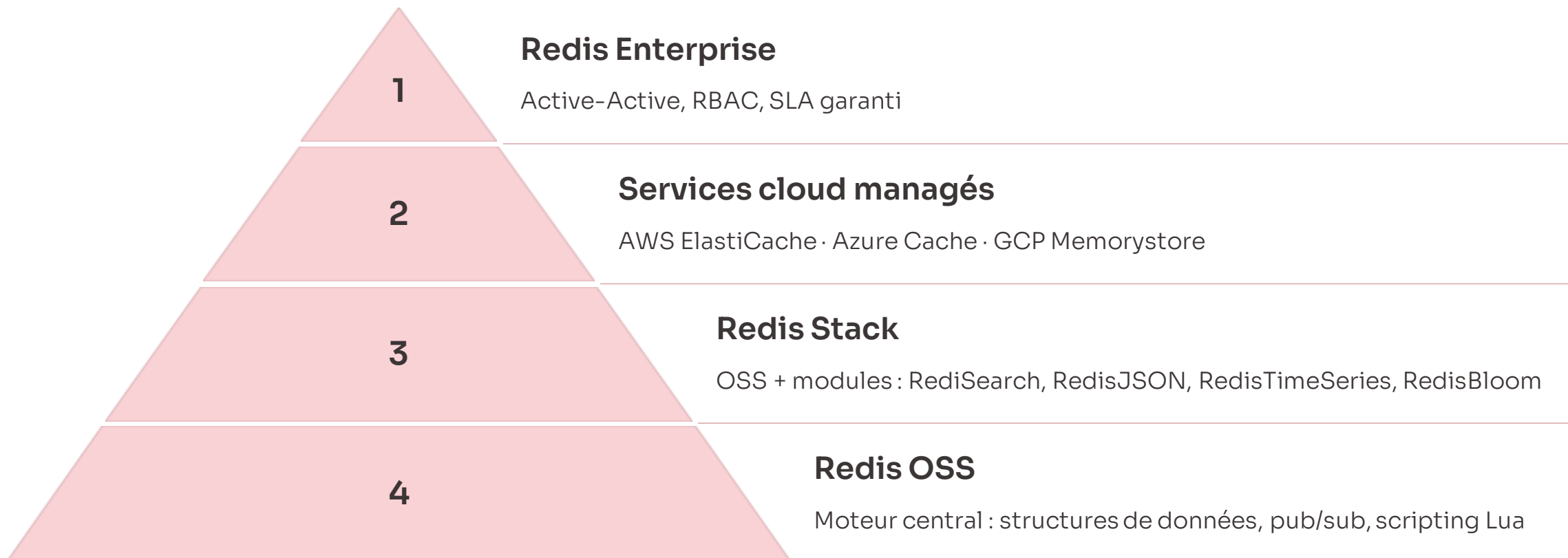


Airbnb

Cache de recherche, gestion des sessions utilisateur et pub/sub pour les notifications instantanées.

L'écosystème Redis en pyramide

Redis évolue en couches superposées, du moteur open-source au service d'entreprise managé à haute disponibilité.



La pyramide illustre une montée en valeur ajoutée : chaque couche hérite des capacités de la précédente tout en ajoutant de la scalabilité, de la sécurité ou de la richesse fonctionnelle.

Licences et historique

2009 – 2024

Redis distribué sous licence **BSD** jusqu'à la version 7.2 — open-source permissive, usage libre.

1

2024 : Fork Valkey

La **Linux Foundation** crée Valkey, fork open-source communautaire, soutenu par AWS, Google, Oracle.

2

3

4

2024 : RSAL/SSPL

Changement de licence vers **RSAL** et **SSPL** — restrictions pour les fournisseurs cloud proposant Redis as-a-service.

Redis 8 : AGPLv3

Retour à une licence open-source avec **AGPLv3**, permettant l'usage applicatif libre sans contrainte.

📌 Pour l'usage applicatif standard (intégration Redis dans votre stack), ces changements de licence n'ont aucune conséquence pratique. La vigilance s'impose uniquement si vous redistribuez Redis as-a-service.

Installation de Redis

Via Docker (recommandé)

```
docker run --name redis \
  -p 6379:6379 \
  redis:7-alpine

# Avec RedisInsight (UI graphique)
# docker-compose.yml
services:
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
  insight:
    image: redis/redisinsight:latest
    ports: ["5540:5540"]
```

Natif Debian / Ubuntu

```
sudo apt update
sudo apt install redis-server

# Démarrage
sudo systemctl start redis
sudo systemctl enable redis
```

✓ Vérification

```
redis-cli PING
# Réponse attendue : PONG

redis-cli INFO server
# Affiche version, OS, uptime...
```

Prise en main de redis-cli

Le client en ligne de commande `redis-cli` est l'outil principal d'interaction, de diagnostic et d'administration de Redis.

 The picture can't be displayed.

Options de connexion

```
redis-cli -h <host> -p <port>  
-a <password> -n <db> — se  
connecter à une instance  
distante avec authentification sur  
une base spécifique.
```

 The picture can't be displayed.

Exploration et diagnostic

`--scan` : itère sur les clés sans bloquer. `--stat` : statistiques temps réel. `--latency` : mesure la latence réseau. `--bigkeys` : identifie les clés volumineuses.

 The picture can't be displayed.

MONITOR — avec précaution

`MONITOR` affiche toutes les commandes en temps réel. Excellent en développement pour déboguer. En production, il peut diviser le débit par deux — à éviter absolument sur des instances sous charge.

Anatomie de redis.conf

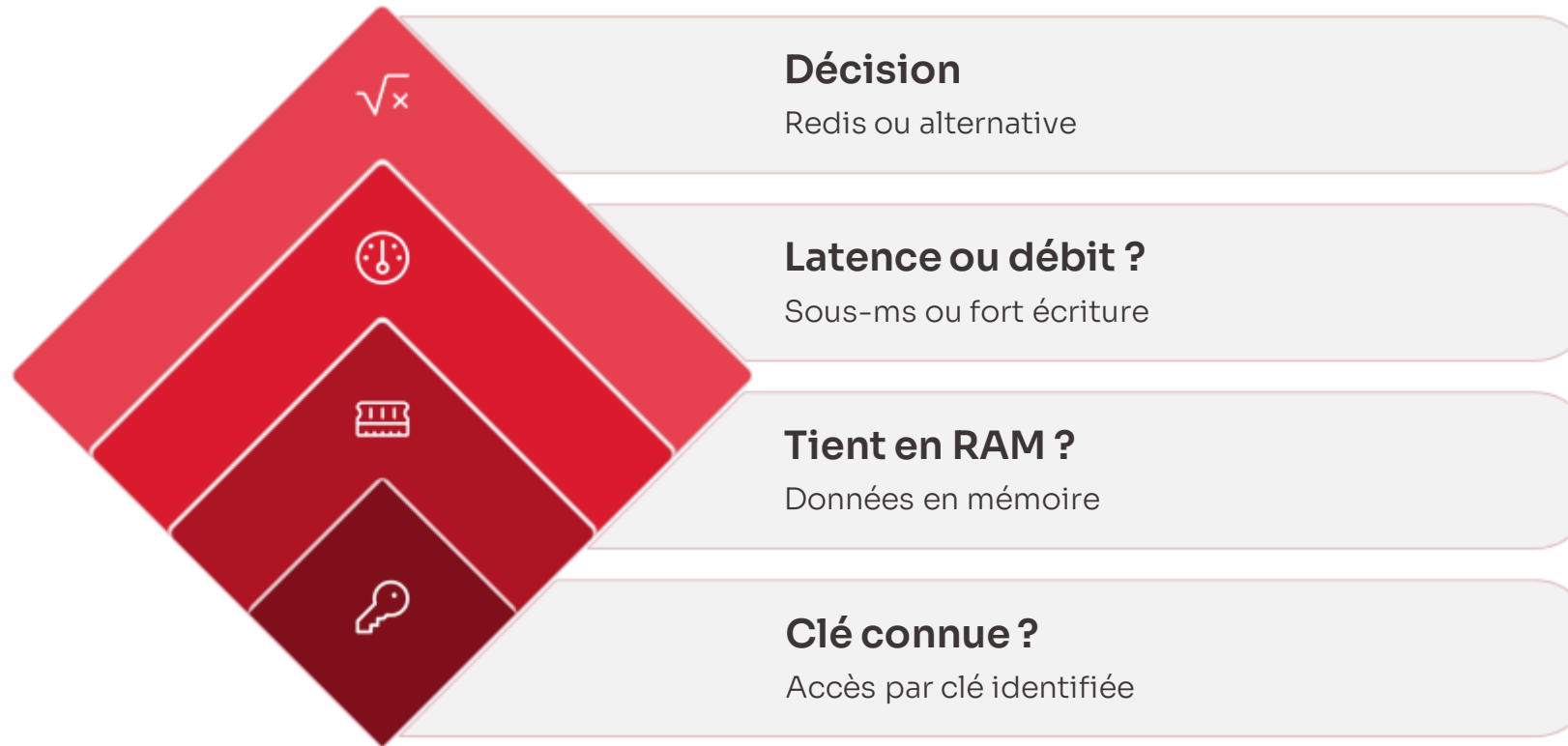
Le fichier de configuration `redis.conf` centralise tous les paramètres de Redis. Les directives clés à maîtriser :

Directive	Rôle	Exemple
<code>bind</code>	Interfaces réseau écoutées	<code>bind 127.0.0.1</code>
<code>protected-mode</code>	Refuse les connexions sans auth	<code>protected-mode yes</code>
<code>requirepass</code>	Mot de passe d'accès	<code>requirepass monSecret</code>
<code>maxmemory</code>	Limite RAM utilisable	<code>maxmemory 256mb</code>
<code>save</code>	Persistance RDB (snapshot)	<code>save 900 1</code>
<code>appendonly</code>	Persistance AOF (log des ops)	<code>appendonly yes</code>

Rechargement à chaud sans redémarrage : `CONFIG GET maxmemory` · `CONFIG SET maxmemory 512mb` · `CONFIG REWRITE` pour persister les changements dans le fichier.

Arbre de décision : Redis ou pas ?

Utilisez ce guide pour décider rapidement si Redis est la bonne solution pour votre besoin.



✓ Clé connue + RAM + latence

Redis est le choix optimal

⚠️ Données > RAM

Envisager Cassandra ou MongoDB

✗ Jointures complexes

PostgreSQL reste irremplaçable

Synthèse du Module 1

Les fondamentaux à retenir avant de passer à la pratique.

1 Redis = vitesse + richesse

In-memory avec structures de données typées — bien plus qu'un simple cache clé-valeur, c'est un serveur de structures manipulables côté serveur.

3 Complémentarité, pas substitution

Redis complète PostgreSQL ou MongoDB — il ne les remplace pas. Il excelle quand la clé d'accès est connue et que la donnée tient en RAM.

2 Trois rôles complémentaires

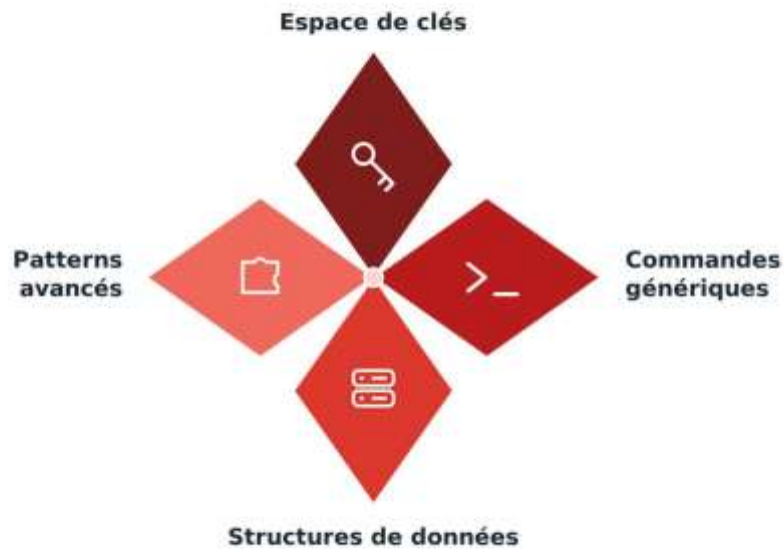
Cache, base de données primaire pour données chaudes, et broker de messages léger. Redis s'intègre dans presque toutes les architectures modernes.

4 Écosystème vivant

OSS → Stack → Cloud managé → Enterprise. Malgré les turbulences de licences, Redis reste la référence — le retour à AGPLv3 avec Redis 8 clarifie la situation.

Module 2 : Clés et Structures de Données Redis

Maîtriser l'espace de clés, les commandes génériques et les cinq structures fondamentales pour concevoir des applications Redis performantes et robustes.



Ce que vous allez apprendre

01

Espace de clés

Conventions de nommage, préfixes par domaine, pièges courants.

02

Commandes génériques

EXISTS, TYPE, DEL, UNLINK, SCAN — et pourquoi KEYS est interdit en prod.

03

Expiration & TTL

EXPIRE, PERSIST, modificateurs Redis 7, pièges KEeptTL.

04

Les 5 structures

String, List, Hash, Set, Sorted Set — cas d'usage et complexités.

05

Arbre de décision & TP

Choisir la bonne structure + mise en pratique redis-cli.

Convention de nommage : les deux-points

Redis n'impose aucun format de clé, mais la convention deux-points est universellement adoptée. Elle structure l'espace de clés de façon lisible et évite les collisions entre domaines applicatifs.

✓ Bonnes pratiques

```
user:1042:panier
session:abc123:data
product:42:stock
order:2024:pending
```

Préfixe par domaine → sous-domaine → identifiant. Lisible, filtrable avec SCAN MATCH.

✗ Anti-patterns

```
u1042p           ← cryptique
verylongkeyname:with:too:many:
  nested:segments ← verbeux
data             ← trop générique
```

Les clés trop courtes sont illisibles en debug. Les clés trop longues consomment de la mémoire inutilement (chaque caractère compte).

📌 Règle d'or : une clé doit être auto-documentée. Un collègue qui la voit en monitoring doit comprendre immédiatement ce qu'elle représente.

EXISTS, TYPE, DEL, RENAME, DBSIZE

Ces commandes s'appliquent à **toutes les structures** quel que soit leur type. Elles forment la boîte à outils de base pour inspecter et gérer l'espace de clés.

Commande	Description	Complexité	Retour
EXISTS key [key...]	Vérifie l'existence (multi-clés)	O(1)	nb de clés trouvées
TYPE key	Retourne le type Redis de la clé	O(1)	string/list/hash...
DEL key [key...]	Suppression synchrone bloquante	O(N)	nb clés supprimées
UNLINK key [key...]	Suppression asynchrone non-bloquante	O(1)*	nb clés supprimées
RENAME key newkey	Renomme une clé atomiquement	O(1)	OK
DBSIZE	Nombre total de clés dans la DB	O(1)	entier

⚠ **PIÈGE** : `DEL` est bloquant. Sur une List de 10 millions d'éléments, il gèle le serveur. Préférez toujours `UNLINK` en production — la mémoire est libérée en arrière-plan.

KEYS vs SCAN : le duel critique

✗ KEYS pattern — JAMAIS en prod

```
127.0.0.1:6379> KEYS user:*  
# Bloque le thread principal  
# pendant toute l'itération  
# → latence spike pour TOUS  
#   les clients connectés
```

KEYS est $O(N)$ et monopolise le serveur. Sur 10M de clés, plusieurs secondes de gel.

✓ SCAN avec curseur — toujours

```
127.0.0.1:6379> SCAN 0 MATCH user:* COUNT 100  
1) "3712" ← curseur suivant  
2) 1) "user:42:panier"  
   2) "user:99:session"  
  
127.0.0.1:6379> SCAN 3712 MATCH user:* COUNT 100  
1) "0" ← itération terminée  
2) ...
```

SCAN est incrémental, non-bloquant. Variantes spécialisées :
HSCAN / SSCAN / ZSCAN.

Le paramètre `COUNT` est une *indication* (pas une garantie) du nombre d'éléments examinés par itération. Recommandé : 100–500 selon la charge.

EXPIRATION

TTL et commandes d'expiration

Redis offre un contrôle fin sur la durée de vie des clés. Comprendre les valeurs de retour de `TTL` et les modificateurs Redis 7 évite des bugs de fuite mémoire classiques.

Commande	Usage	Unité
<code>EXPIRE key seconds</code>	Définit une expiration en secondes	secondes
<code>PEXPIRE key ms</code>	Expiration en millisecondes (précision haute)	ms
<code>EXPIREAT key timestamp</code>	Expiration à un timestamp Unix absolu	Unix s
<code>TTL key</code>	Temps restant : -1 = persistante, -2 = n'existe pas	secondes
<code>PERSIST key</code>	Supprime l'expiration → clé permanente	—

NX

Expire seulement si aucun TTL n'existe encore

XX

Expire seulement si un TTL est déjà défini

GT

Expire seulement si le nouveau TTL est **plus grand** (prolonge sans raccourcir)

LT

Expire seulement si le nouveau TTL est **plus petit**

KEEPTTL : le modificateur oublié

✗ Fuite mémoire silencieuse

```
127.0.0.1:6379> SET session:u42 "data" EX 3600
OK
# ... 1800 secondes plus tard ...
127.0.0.1:6379> SET session:u42 "new_data"
OK
127.0.0.1:6379> TTL session:u42
(integer) -1 ← TTL effacé ! Fuite mémoire.
```

Un `SET` sans `KEEPTTL` réinitialise le TTL à -1 (persistant). La clé ne sera jamais supprimée automatiquement.

✓ Avec KEEPTTL

```
127.0.0.1:6379> SET session:u42 "data" EX 3600
OK
# ... 1800 secondes plus tard ...
127.0.0.1:6379> SET session:u42 "new_data" KEEPTTL
OK
127.0.0.1:6379> TTL session:u42
(integer) 1800 ← TTL conservé ✓
```

`KEEPTTL` préserve l'expiration existante lors d'une mise à jour de valeur. Indispensable pour les sessions et caches.

Les deux mécanismes d'expiration coexistent : `lazy` (la clé est supprimée à l'accès si expirée) et `active` (un processus de fond parcourt aléatoirement les clés expirées).

String : le type de base universel

Le type **String** est binaire, jusqu'à **512 Mo**. Il stocke du texte, du JSON, des images sérialisées, ou des entiers pour les compteurs atomiques.

Commandes essentielles

Commande	O()	Usage
SET / GET	O(1)	Base
MSET / MGET	O(N)	Batch
APPEND	O(1)	Concat
GETRANGE	O(N)	Sous-chaîne
INCR/INCRBY	O(1)	Compteur
INCRBYFLOAT	O(1)	Float

Options de SET (Redis 7)

```
127.0.0.1:6379> SET clé valeur
EX 3600      # TTL secondes
PX 3600000   # TTL ms
NX           # seulement si absente
XX           # seulement si présente
KEEPTTL      # préserver TTL
GET          # retourner ancienne val
```

MGET : gain réseau

```
127.0.0.1:6379> MGET k1 k2 k3 ... k100
# 100 clés en 1 aller-retour réseau
```



Pattern verrou distribué : `SET resource:lock "owner" NX EX 30` — atomique, pas de race condition.

INCR : éliminer la race condition

✗ Approche naïve (race condition)

```
# Thread A et Thread B simultanément :  
val = GET compteur    # A lit 5, B lit 5  
val = val + 1         # A calcule 6, B calcule 6  
SET compteur val      # A écrit 6, B écrit 6  
# Résultat : 6 au lieu de 7 !
```

Le triplet lire-modifier-écrire n'est pas atomique. Deux threads concurrents perdent une incrémentation.

✓ INCR atomique

```
127.0.0.1:6379> SET compteur 5  
OK  
127.0.0.1:6379> INCR compteur  
(integer) 6  
127.0.0.1:6379> INCRBY compteur 10  
(integer) 16  
127.0.0.1:6379> INCRBYFLOAT prix 1.50  
"17.50"
```

INCR est une opération atomique côté serveur. Impossible d'avoir une race condition : Redis est mono-thread sur les commandes.

Cas d'usage : compteurs de vues, rate limiting simple, génération d'IDs uniques, statistiques temps réel.

List : liste chaînée O(1) aux extrémités

La **List** Redis est une liste doublement chaînée. Les opérations aux extrémités sont O(1), ce qui la rend idéale pour les files et journaux bornés.

Commandes clés

Commande	O()	Note
LPUSH / RPUSH	O(1)	Push gauche/droite
LPOP / RPOP	O(1)	Pop gauche/droite
LRANGE key 0 -1	O(N)	Plage d'éléments
LLEN	O(1)	Longueur
LTRIM	O(N)	Tronquer
BLPOP / BRPOP	O(1)	Bloquant
LMOVE	O(1)	Atomique

Pattern : journal des N derniers

```
127.0.0.1:6379> LPUSH events:log "event_new"
(integer) 1
127.0.0.1:6379> LTRIM events:log 0 99
OK
# Garde toujours les 100 derniers
# événements, supprime les anciens
```

File fiable avec LMOVE

```
127.0.0.1:6379> LMOVE attente en_cours LEFT LEFT
"task:42"
# Atomique : pas de perte si crash
# LREM en_cours 1 "task:42" après succès
```

 BLPOP/BRPOP nécessitent une **connexion dédiée** côté client — ne pas partager avec d'autres commandes car la connexion reste suspendue jusqu'à l'arrivée d'un élément.

Hash : le dictionnaire champ/valeur

Le **Hash** stocke un dictionnaire de paires champ→valeur. Idéal pour représenter des objets (utilisateur, produit) sans sérialiser en JSON.

Commandes essentielles

Commande	O()	Usage
HSET key f1 v1 f2 v2	O(N)	Multi-champs
HGET key field	O(1)	Un champ
HMGET key f1 f2...	O(N)	Multi-champs
HGETALL key	O(N)	Tous les champs
HDEL key field	O(1)	Supprime un champ
HINCRBY / HSETNX	O(1)	Incrémente / NX

Exemple objet utilisateur

```
127.0.0.1:6379> HSET user:1042
  nom "Alice"
  email "alice@ex.com"
  age 29
(integer) 3
127.0.0.1:6379> HMGET user:1042 nom email
1) "Alice"
2) "alice@ex.com"
```

⚠️ PIÈGE : HGETALL sur gros Hash

```
# Hash avec 100 000 champs :
HGETALL bigkey ← bloquant O(N)
# Préférer HMGET (champs ciblés)
# ou HSCAN pour itérer
```

📌 **Hash sharding** : stockez des objets similaires dans un seul Hash (`user:bucket:42` → `{1000: data, 1001: data...}`) pour profiter de l'encodage `listpack` compact et réduire la mémoire drastiquement.

Set : membres uniques sans ordre

Le Set garantit l'unicité des membres avec des opérations d'appartenance $O(1)$. Parfait pour les tags, relations sociales, unicité de traitement.

Commandes

Commande	$O()$	Usage
SADD / SREM	$O(1)$	Ajoute/Retire
SISMEMBER	$O(1)$	Appartenance
SMEMBERS	$O(N)$	Tous (grand set)
SCARD	$O(1)$	Cardinalité
SPOP / SRANDMEMBER	$O(1)$	Tirage
SINTER / SUNION / SDIFF	$O(N)$	Ensemble

Opérations ensemblistes natives

```
127.0.0.1:6379> SADD tags:article:1 redis nosql cache
(integer) 3
127.0.0.1:6379> SADD tags:article:2 redis database

# Diagramme de Venn natif :
127.0.0.1:6379> SINTER tags:article:1 tags:article:2
1) "redis"

127.0.0.1:6379> SUNIONSTORE
tags:all tags:article:1 tags:article:2
(integer) 4
```

Variantes `STORE` : résultat persisté dans une nouvelle clé.

Cas d'usage typiques : unicité d'emails traités, suivi des utilisateurs connectés, relations suiveurs/suivis, tirages au sort équitables avec SPOP.

Sorted Set : membres uniques + score trié

Le Sorted Set associe un score flottant à chaque membre. L'ensemble est automatiquement trié par score. Complexité $O(\log N)$ pour la plupart des opérations.

Commandes avancées

Commande	O()	Note
ZADD key score member	$O(\log N)$	Options NX/XX/GT/LT
ZINCRBY key incr member	$O(\log N)$	Incrémente score
ZSCORE key member	$O(1)$	Score d'un membre
ZRANGE ... BYSCORE	$O(\log N + M)$	Plage par score
ZRANK / ZREVRANK	$O(\log N)$	Position
ZCOUNT / ZREMRANGEBYSCORE	$O(\log N)$	Comptage/suppr ession
ZUNIONSTORE WEIGHTS	$O(N \log N)$	Union pondérée

Leaderboard en temps réel

```
127.0.0.1:6379> ZADD leaderboard GT 1500 "Alice"
(integer) 1
127.0.0.1:6379> ZADD leaderboard GT 2200 "Bob"
(integer) 1
# GT : met à jour seulement si
# nouveau score > score actuel

127.0.0.1:6379> ZRANGE leaderboard 0 -1
REV WITHSCORES
1) "Bob" 2) "2200"
3) "Alice" 4) "1500"

127.0.0.1:6379> ZREVRANK leaderboard "Alice"
(integer) 1 ← 2ème place (0-indexé)
```

Autocomplétion, Rate Limiting et Files de priorité

The picture can't be displayed.

Autocomplétion lexicographique

Avec des scores identiques (0), `ZRANGE key [préfixe [préfixe\xff BYLEX` retourne tous les membres commençant par le préfixe — moteur de recherche instantané.

```
127.0.0.1:6379> ZADD dict 0
"redis" 0 "redisson" 0 "react"
127.0.0.1:6379> ZRANGE dict
"[re" "[re\xff" BYLEX
1) "react" 2) "redis" 3)
"redisson"
```

The picture can't be displayed.

Rate limiting fenêtre glissante

Stocker chaque requête avec son timestamp comme score. Compter les éléments dans la fenêtre, supprimer les anciens — précis à la milliseconde.

```
127.0.0.1:6379> ZADD rl:user:42
1700000000 "req1"
127.0.0.1:6379>
ZREMRANGEBYSCORE rl:user:42 0
(now-60s)
127.0.0.1:6379> ZCARD
rl:user:42 ← nb requêtes/min
```

The picture can't be displayed.

File de priorité

Score = priorité ou timestamp d'échéance. `ZPOPMIN` dépile toujours la tâche la plus urgente, atomiquement, sans race condition.

```
127.0.0.1:6379> ZADD tasks 1
"task:urgent"
127.0.0.1:6379> ZADD tasks 10
"task:low"
127.0.0.1:6379> ZPOPMIN tasks
1) "task:urgent" 2) "1"
```

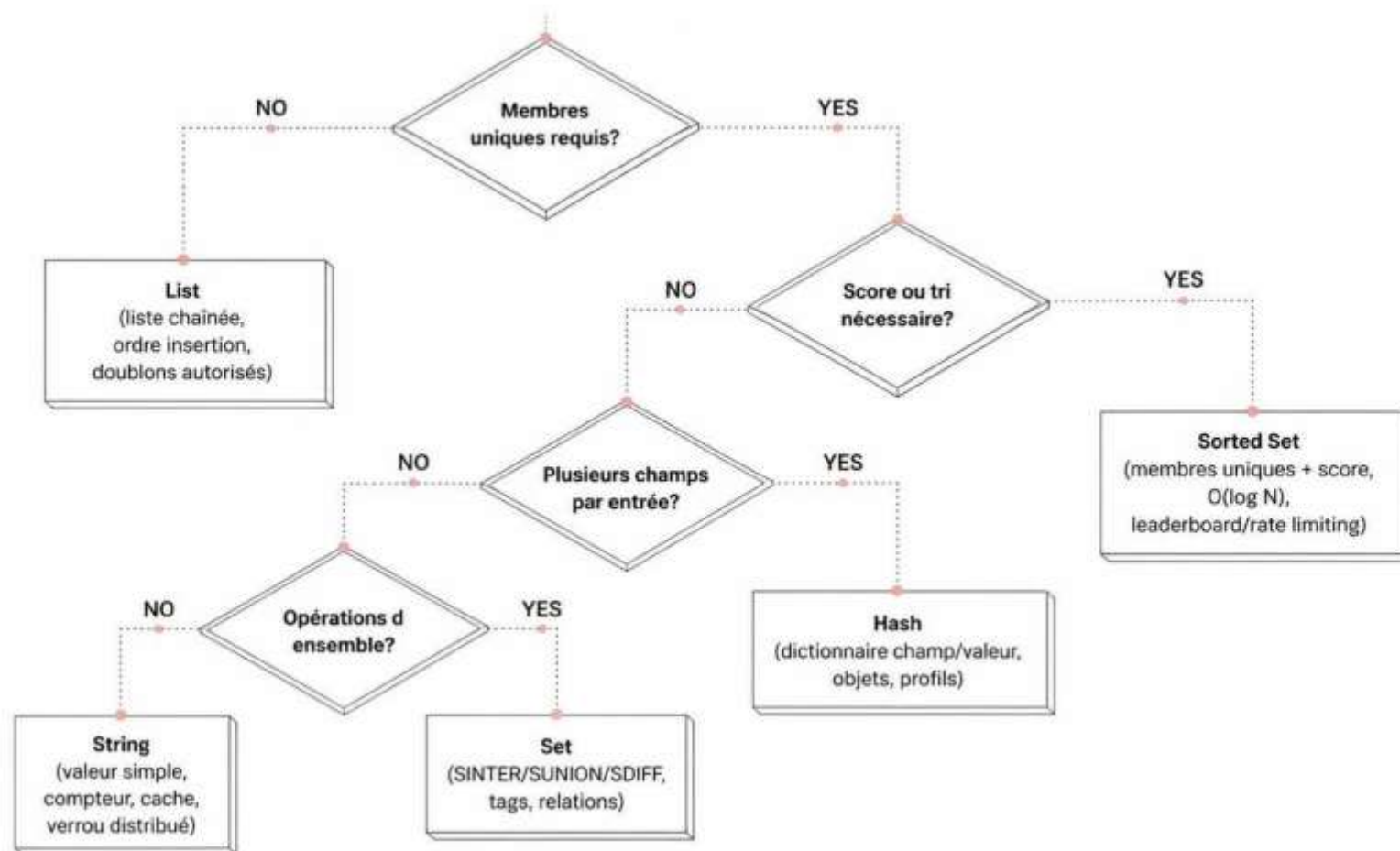
Tableau comparatif des 7 structures

Structure	Unicité	Ordre	Complexité clé	Cas d'usage typiques
String	N/A	N/A	O(1) GET/SET	Cache, compteurs, verrou distribué, JSON
List	Non	Insertion	O(1) extrémités	File de messages, journal borné, pipeline
Hash	Champs oui	Non	O(1) par champ	Objets, sessions, profils utilisateurs
Set	Oui	Non	O(1) appart.	Tags, unicité, relations sociales, tirages
Sorted Set	Oui	Par score	O(log N)	Leaderboard, rate limiting, autocomplétion
Bitmap	N/A	Par offset	O(1) SETBIT/GETBIT	Présence quotidienne, rétention, feature flags
HyperLogLog	Estimée ($\pm 0,81\%$)	Non	O(1) PFADD/PFCOUNT	Visiteurs uniques, comptage (12 Ko max)

❏ Mémoire : préférez les structures compactes (`listpack`) en restant sous les seuils `hash-max-listpack-entries` (128 par défaut) et `hash-max-listpack-value` (64 octets). Au-delà, Redis bascule sur une structure plus lourde.

Quelle structure choisir ?

Suivez cet arbre de décision pour choisir la structure Redis adaptée à votre besoin. Chaque branche correspond à une caractéristique fonctionnelle clé.



Résumé de l'arbre de décision

1

Membres uniques ? → NON

List — ordre d'insertion préservé, doublons autorisés. Idéal pour files de messages, journaux bornés.

2

Score / tri ? → OUI

Sorted Set — membres uniques triés par score flottant. Leaderboard, rate limiting, autocomplétion.

3

Plusieurs champs ? → OUI

Hash — dictionnaire champ→valeur. Représente des objets sans sérialisation.

4

Opérations d'ensemble ? → OUI

Set — unicité + SINTER/SUNION/SDIFF. Tags, relations sociales, tirages.

5

Valeur simple ? → OUI

String — cache, compteur atomique, verrou distribué (NX EX).

DEMO : Système de sessions avec Hash

Objectif : créer et gérer des sessions utilisateurs en utilisant les Hashes Redis avec expiration automatique.

```
127.0.0.1:6379> HSET session:abc123
  user_id 1042
  username "Alice"
  role "admin"
  last_seen "2024-01-15T10:30:00Z"
(integer) 4

127.0.0.1:6379> EXPIRE session:abc123 3600
(integer) 1

# Vérifier la session
127.0.0.1:6379> HMGET session:abc123 user_id username role
1) "1042"
2) "Alice"
3) "admin"

# Mettre à jour sans perdre le TTL
127.0.0.1:6379> HSET session:abc123 last_seen "2024-01-15T11:00:00Z"
127.0.0.1:6379> EXPIRE session:abc123 3600 GT
(integer) 1
# GT : prolonge si nouveau TTL > TTL restant

127.0.0.1:6379> TTL session:abc123
(integer) 3600

# Lister les sessions actives (SCAN, pas KEYS!)
127.0.0.1:6379> SCAN 0 MATCH session:* COUNT 100
1) "0"
2) 1) "session:abc123"
```

DEMO : Leaderboard en temps réel

Objectif : construire un leaderboard de jeu avec Sorted Set — mise à jour de scores, classement, et pagination.

```
127.0.0.1:6379> ZADD leaderboard:game1 GT 1500 "Alice"
(integer) 1
127.0.0.1:6379> ZADD leaderboard:game1 GT 2200 "Bob"
(integer) 1
127.0.0.1:6379> ZADD leaderboard:game1 GT 1800 "Charlie"
(integer) 1

# Alice améliore son score
127.0.0.1:6379> ZADD leaderboard:game1 GT 2500 "Alice"
(integer) 0 ← 0 = membre existant, mis à jour

# Top 3 (ordre décroissant)
127.0.0.1:6379> ZRANGE leaderboard:game1 0 2 REV WITHSCORES
1) "Alice" 2) "2500"
3) "Bob" 4) "2200"
5) "Charlie" 6) "1800"

# Rang d'un joueur
127.0.0.1:6379> ZRANK leaderboard:game1 "Bob" REV
(integer) 1 ← 2ème place

# Joueurs entre 2000 et 3000 points
127.0.0.1:6379> ZCOUNT leaderboard:game1 2000 3000
(integer) 2 ← Alice et Bob

# Incrémenter atomiquement (sans race condition)
127.0.0.1:6379> ZINCRBY leaderboard:game1 300 "Charlie"
"2100"
```

DEMO : File de messages fiable avec LMOVE

Objectif : implémenter une file de tâches fiable (reliable queue) qui garantit qu'aucune tâche n'est perdue en cas de crash du worker.

```
127.0.0.1:6379> RPUSh queue:attente "task:email:101" "task:sms:102" "task:push:103"
(integer) 3

# Worker déplace atomiquement vers "en_cours"
127.0.0.1:6379> LMOVE queue:attente queue:en_cours LEFT LEFT
"task:email:101"

# Worker traite la tâche...
# Succès → supprimer de en_cours
127.0.0.1:6379> LREM queue:en_cours 1 "task:email:101"
(integer) 1

# Surveiller les tâches bloquées (ex : timeout)
127.0.0.1:6379> LRANGE queue:en_cours 0 -1
1) (empty array) ← OK, rien en attente

# Journal des N derniers événements (pattern borné)
127.0.0.1:6379> LPUSH events:recent "evt:connexion:Alice"
(integer) 1
127.0.0.1:6379> LTRIM events:recent 0 99
OK ← garde seulement les 100 derniers
```

📘 En cas de crash du worker, les tâches restent dans `queue:en_cours`. Un processus de monitoring peut les republier dans `queue:attente` après un timeout.

Les 5 pièges à éviter absolument

1 KEYS en production

Bloque le thread principal pendant toute l'itération. Toujours remplacer par `SCAN` avec curseur et `MATCH/COUNT`.

2 DEL sur grosse structure

`DEL` est synchrone et bloquant. Utiliser `UNLINK` qui libère la mémoire en arrière-plan sans impacter les autres clients.

3 SET sans KEEPTTL

Un `SET` ordinaire efface le TTL existant → la clé devient persistante et ne sera jamais supprimée : fuite mémoire silencieuse.

4 HGETALL sur gros Hash

$O(N)$ sur un Hash de 100K champs est bloquant. Préférer `HMGET` sur les champs ciblés, ou `HSCAN` pour itérer.

5 Race condition lire-modifier-écrire

Ne jamais faire `GET+calcul+SET` pour des compteurs. Utiliser `INCR/INCRBY` ou `ZADD GT` qui sont atomiques par nature.

Complexités à retenir

$O(1)$ — Opérations constantes

- GET, SET, EXISTS, TYPE — String
- LPUSH, RPUSH, LPOP, RPOP, LLEN — List
- HGET, HSET (1 champ), HINCRBY — Hash
- SADD, SREM, SISMEMBER, SCARD — Set
- ZSCORE — Sorted Set
- UNLINK (déclenchement), EXPIRE — Génériques

$O(\log N)$ — Opérations arbre

- ZADD, ZINCRBY, ZRANK, ZCOUNT — Sorted Set
- ZRANGE BYSCORE — Sorted Set

$O(N)$ — Proportionnel à la taille

- MGET, MSET — String (N = nb clés)
- LRANGE, LTRIM — List
- HGETALL, HMGET — Hash
- SMEMBERS, SINTER, SUNION — Set
- DEL — Génériques (⚠ bloquant)
- KEYS — 🚫 Interdit en production

Points clés du Module 2

❏ Espace de clés

Convention `domaine:id:sous-domaine`. SCAN toujours, jamais KEYS. UNLINK plutôt que DEL sur les grosses structures.

❏ Expiration

Modificateurs GT/LT/NX/XX (Redis 7). KEEPTTL obligatoire sur les SET de mise à jour. TTL -1 = persistant, -2 = inexistant.

📦 5 Structures

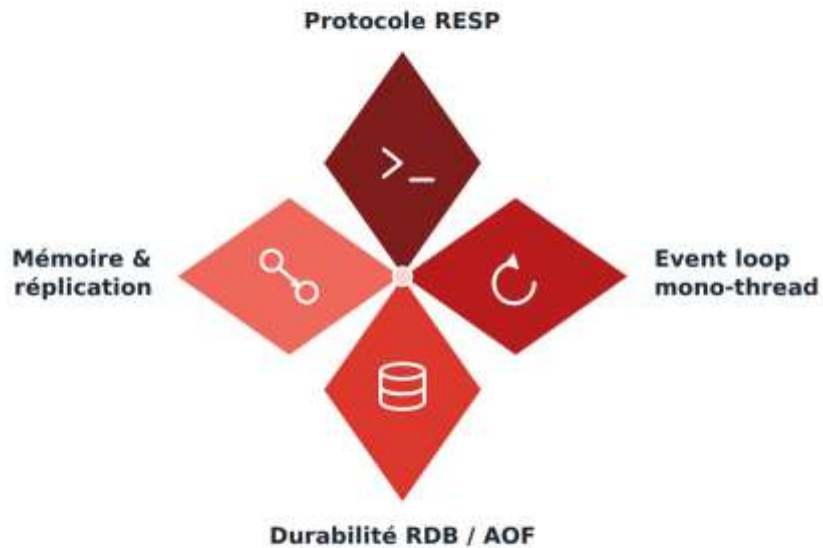
String → compteur atomique. List → file fiable. Hash → objets. Set → unicité + ensembles. Sorted Set → tri + score $O(\log N)$.

🔑 Arbre de décision

Unique ? → score ? → Sorted Set. Plusieurs champs ? → Hash. Opérations d'ensemble ? → Set. Ordre insertion ? → List. Sinon → String.

Prochain module : Persistance, réplication et haute disponibilité — RDB, AOF, Redis Sentinel et Cluster.

Module 3 : L'Architecture Redis



Plongée technique au cœur du moteur Redis — protocole RESP, event loop mono-thread, durabilité RDB/AOF, gestion mémoire et réplication. Ce module fournit les fondations architecturales indispensables pour concevoir, configurer et opérer Redis en production.

Plan du Module

Un parcours progressif des couches internes de Redis, du protocole réseau jusqu'à la réplication.

01

Protocole RESP

Types, échanges, RESP2 vs RESP3

02

Atomicité & Event Loop

Mono-thread, multiplexage I/O

03

Durabilité

RDB, AOF, mode mixte

04

Gestion Mémoire

Éviction, fragmentation, monitoring

05

Réplication

Master-replica, PSYNC, cohérence

Vue d'Ensemble Interne

Redis est architecturé autour de quelques principes fondamentaux qui expliquent ses performances exceptionnelles : un unique thread de traitement des commandes, une event loop non bloquante, et des structures de données en mémoire optimisées.



Couche Réseau

Protocole RESP via TCP. Multiplexage des connexions clientes par l'event loop `ae`.



Couche Persistance

Processus `fork()` enfant pour RDB/AOF. Isolation du thread principal via copy-on-write.



Moteur de Commandes

Thread unique pour l'exécution des commandes. Atomicité garantie, pas de verrou applicatif.



Gestionnaire Mémoire

Allocateur `jemalloc`, politique d'éviction configurable, monitoring via `INFO memory`.

Protocole RESP : Les Types de Données

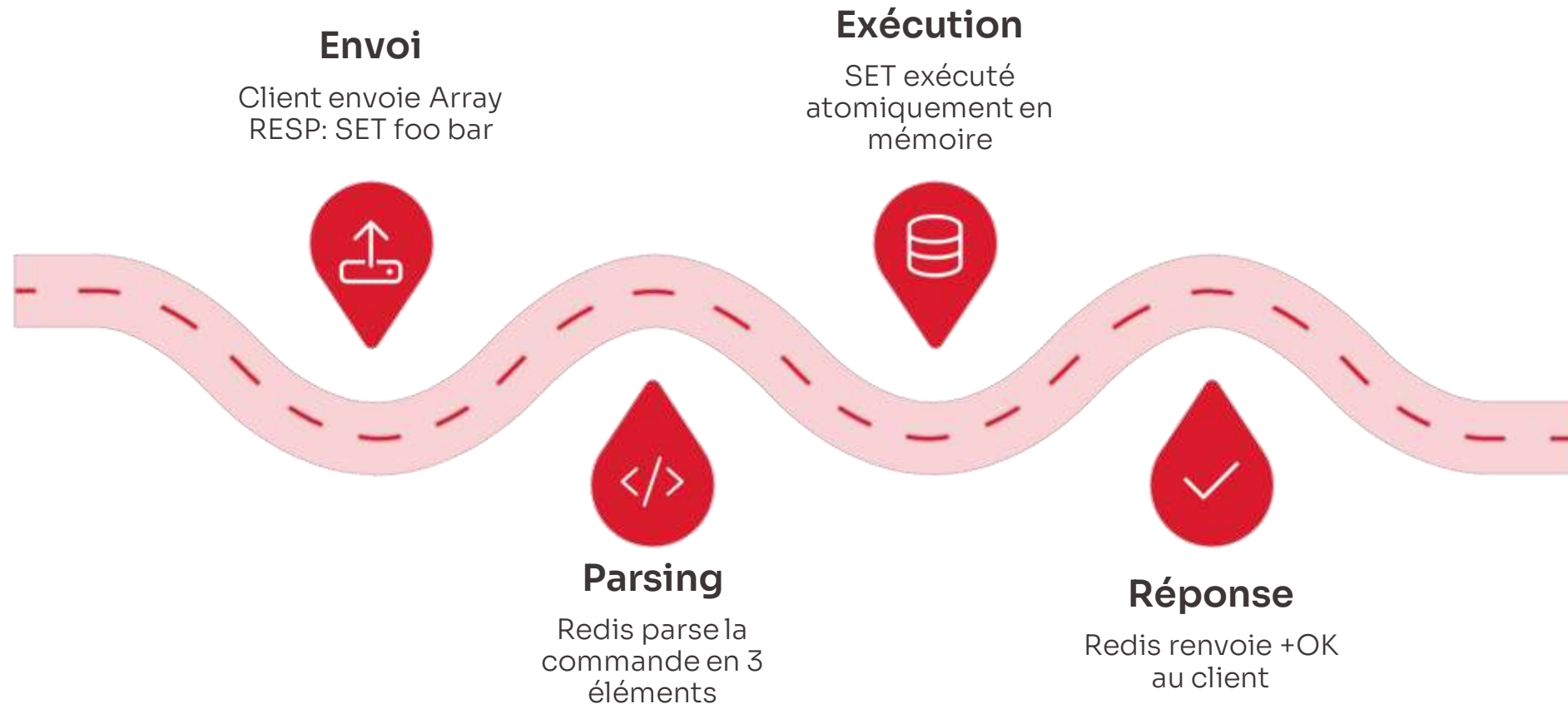
RESP (REdis Serialization Protocol) définit 5 types primitifs reconnaissables au premier octet. Ce protocole texte est conçu pour être simple à parser et lisible par un humain dans un terminal.

Préfixe	Type	Exemple
+	Simple String	+OK\r\n
-	Error	-ERR unknown command\r\n
:	Integer	:1000\r\n
\$	Bulk String	\$6\r\nfoobar\r\n
*	Array	*2\r\n\$3\r\nGET\r\n\$3\r\nkey\r\n

 Les commandes Redis sont envoyées sous forme d'Arrays de Bulk Strings. La réponse dépend du type de retour de la commande.

RESP : Séquence Requête / Réponse

Illustration d'un échange complet `SET foo bar` entre un client et Redis. Chaque message respecte le formatage RESP avec terminaison `\r\n`.



Le client formate toujours ses commandes en tableau RESP. Le serveur peut répondre avec n'importe quel type primitif selon la commande exécutée.

RESP2 vs RESP3

Redis 6 a introduit RESP3, négocié via la commande `HELLO` 3. RESP3 apporte des types enrichis et des capacités inédites côté client.

RESP2 (historique)

- 5 types primitifs uniquement
- Hashes retournés comme arrays plates
- Pas de push serveur natif
- Compatible tous clients existants

RESP3 (Redis 6+)

- Types enrichis: Map, Set, Double, Bool, Blob Error
- Hashes retournés comme Maps nommées
- **Push messages** : invalidations Pub/Sub natives
- **Client-side caching** : tracking des clés lues, invalidation automatique par le serveur

Le **client-side caching** de RESP3 permet de mettre en cache côté application et de recevoir une notification d'invalidation dès qu'une clé est modifiée — sans polling.

Atomicité & Modèle Mono-Thread

Chaque commande Redis est atomique par conception : aucune autre commande ne peut s'intercaler pendant son exécution. Ce modèle simplifie radicalement la gestion des compteurs et des verrous distribués.

 The picture can't be displayed.

Garantie par commande

`INCR`, `GETSET`, `SETNX` — toutes les opérations sont atomiques. Pas de race condition possible sur une seule commande, même depuis 100 clients concurrents.

 The picture can't be displayed.

Compteurs & Verrous

`INCR counter` est safe sans transaction. Pour les verrous distribués, `SET lock NX EX 30` combine création conditionnelle et TTL en une seule opération atomique.

 The picture can't be displayed.

I/O Threads depuis Redis 6

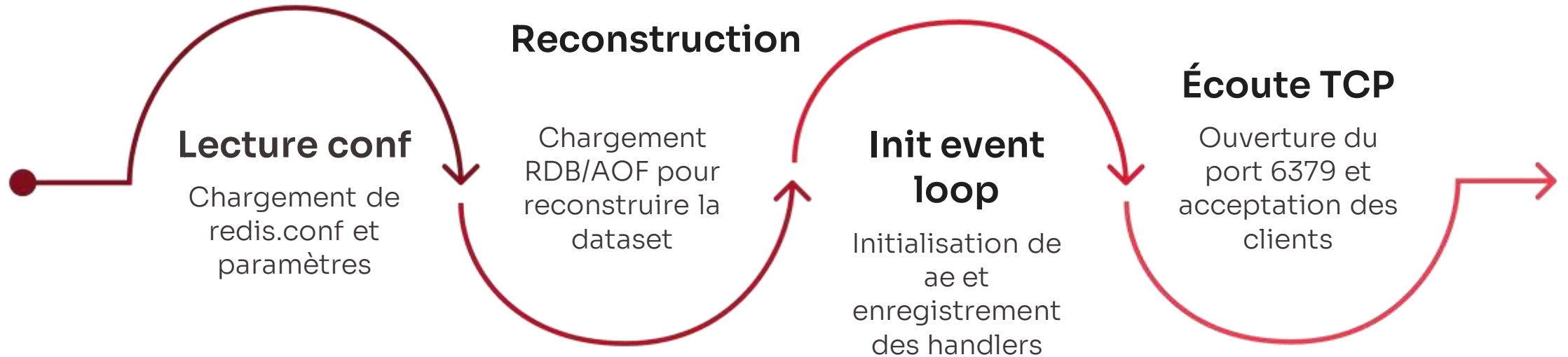
Redis 6 introduit des threads dédiés à la lecture/écriture réseau (`io-threads`). L'exécution des commandes reste mono-thread. Gain : latence réduite sur les payloads volumineux.



Les transactions `MULTI/EXEC` et les scripts Lua offrent l'atomicité sur un bloc de commandes.

Processus de Démarrage

Redis suit une séquence d'initialisation bien définie à chaque démarrage. La comprendre est essentielle pour diagnostiquer les problèmes de démarrage et optimiser le temps de disponibilité.



Si RDB et AOF sont tous deux activés, Redis charge prioritairement le fichier AOF car il garantit une moindre perte de données.

Event Loop (ae) : Le Cœur Performant

L'event loop `ae` (Async Event) est le moteur non-bloquant de Redis. Elle orchestre toutes les entrées/sorties et les timers sans jamais bloquer sur une opération.

Multiplexage I/O

Redis utilise `epoll` (Linux) ou `kqueue` (BSD/macOS) pour surveiller simultanément des milliers de descripteurs de fichiers sans thread par connexion.

Deux types d'événements

- File events : données disponibles sur un socket client → lecture RESP, exécution de commande, écriture de la réponse
- Time events : tâches périodiques (expiration des clés, statistiques, appel de `serverCron` toutes les 100ms)

Pourquoi c'est performant ?

- Zéro syscall bloquant
- Pas de context switch entre threads
- Cache CPU optimal (données chaudes)
- 100 000+ ops/sec sur un seul cœur

❏ L'event loop est la raison pour laquelle les commandes longues (ex: `KEYS *`) bloquent TOUT Redis.

Durabilité RDB : Snapshots

RDB (Redis Database) sauvegarde un snapshot binaire complet de la dataset à intervalles configurables. C'est le mécanisme le plus simple et le plus compact.

Configuration

`save 900 1` — snapshot si ≥ 1 écriture en 15 min

`save 300 10` — ou ≥ 10 écritures en 5 min

`BGSAVE` — déclenche manuellement

Fork + Copy-on-Write

Redis `fork()` un processus enfant qui écrit le RDB. Le parent continue à servir les requêtes. Les pages mémoire modifiées sont copiées (CoW) — prévoir 50% de RAM libre.

Avantages & Limites

- ✓ Fichier compact binaire
- ✓ Restauration très rapide
- ✗ Perte de données = plusieurs minutes (entre deux snapshots)

Durabilité AOF : Append-Only File

AOF (Append Only File) journalise chaque commande d'écriture en texte. C'est le mécanisme qui offre la meilleure durabilité, au prix d'une empreinte disque plus importante.

 The picture can't be displayed.



appendfsync always

Flush disque à chaque commande. Durabilité maximale, performance dégradée. Réservé aux cas critiques.

 The picture can't be displayed.



appendfsync everysec

Recommandé. Flush toutes les secondes. Perte max = ~1 seconde de données. Bon compromis perfo/durabilité.

 The picture can't be displayed.



appendfsync no

Délègue le flush à l'OS. Performance maximale, durabilité imprévisible. Déconseillé en production.

 The picture can't be displayed.



BGREWRITEAOF

Compacte le fichier AOF en réécrivant l'état courant. Redis 7 introduit l'AOF multi-parts avec préambule RDB intégré.

RDB vs AOF vs Mode Mixte

Critère	RDB	AOF	Mixte
Performance	Haute	Bonne	Bonne
Perte de donnéesmax	Minutes	~1 seconde	~1 seconde
Taille fichier	Compact binaire	Verbeux texte	Optimisée
Tempsde restauration	Très rapide	Lent	Rapide
Cas d'usage	Backups / DR	Production critique	Recommandé Redis 4+

📄 Stratégie recommandée : activer les deux avec `aof-use-rdb-preamble yes`. Pour un cache pur, désactiver la persistance est légitime — mais documenter le risque de *cache stampede* au redémarrage.

Sauvegarde & Restauration à Chaud

Opérer Redis en production nécessite des procédures de backup non-intrusives. Redis fournit les outils natifs pour sauvegarder et vérifier l'intégrité des fichiers sans interruption de service.

1

Déclencher BGSAVE

Lancer `BGSAVE` manuellement ou vérifier `LASTSAVE` pour connaître l'horodatage du dernier snapshot réussi.

2

Copier le fichier

Attendre la fin du snapshot (`INFO persistence → rdb_bgsave_in_progress: 0`), puis copier `dump.rdb` vers le stockage cible.

3

Vérifier l'intégrité

`redis-check-rdb dump.rdb` et `redis-check-aof appendonly.aof` détectent les corruptions avant restauration.

En cas de fichier AOF corrompu, `redis-check-aof --fix` tronque le fichier au dernier point cohérent. Toujours tester la restauration en environnement de staging.

Gestion Mémoire : Monitoring

Comprendre la consommation mémoire de Redis est indispensable pour éviter les évictions surprises et les dégradations de performance liées à la fragmentation.

Commandes clés

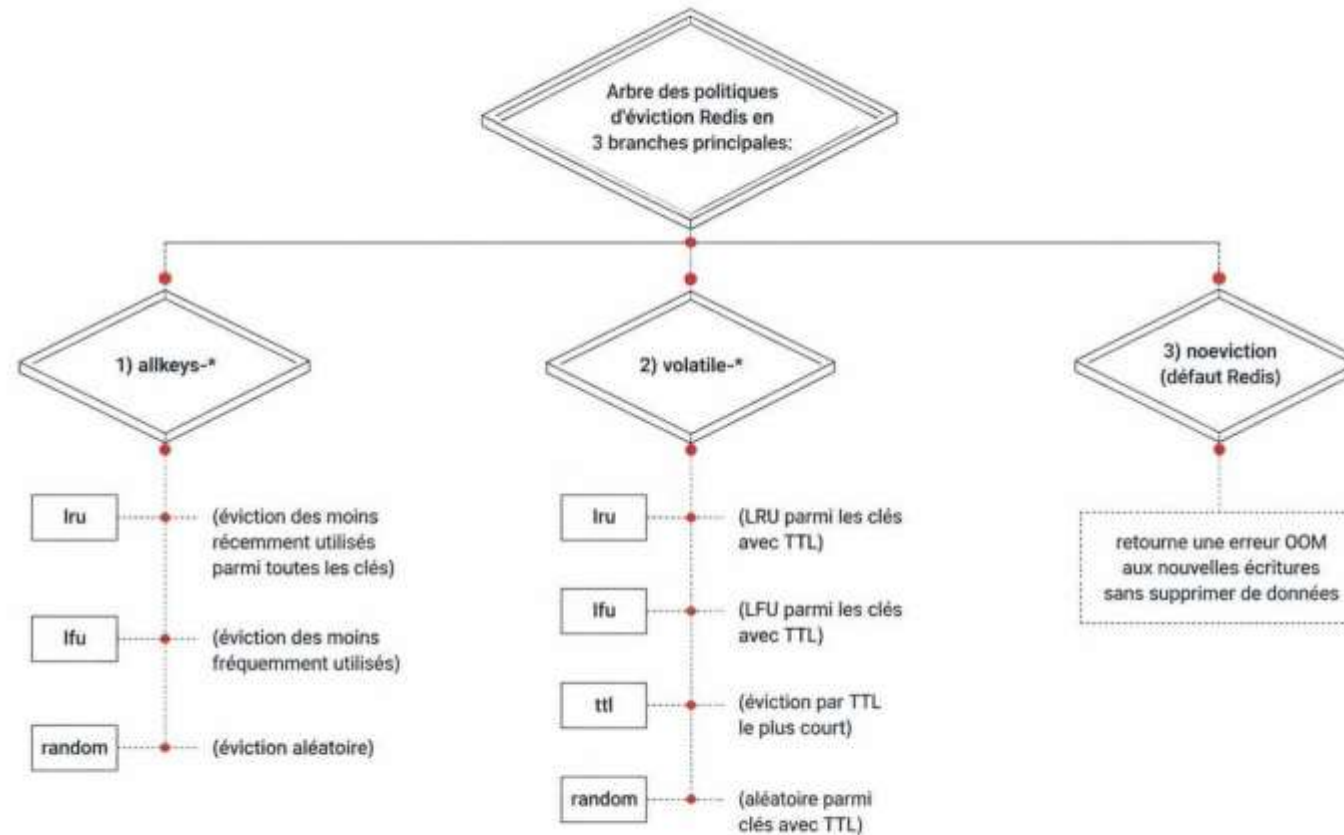
- `INFO memory` — vue complète : `used_memory`, `used_memory_rss`, ratio de fragmentation
- `MEMORY USAGE key` — taille précise d'une clé en bytes
- `MEMORY DOCTOR` — diagnostic automatique avec recommandations
- `redis-cli --bigkeys` — détection des grandes clés

Indicateurs à surveiller

- `mem_fragmentation_ratio > 1.5` → fragmentation excessive, envisager `MEMORY PURGE`
- `used_memory_rss` vs `used_memory` → écart = overhead allocateur
- `maxmemory` → fixer à 60–75% de la RAM disponible

Politiques d'Éviction Mémoire

Quand `maxmemory` est atteint, Redis applique la politique d'éviction configurée. Le choix dépend du rôle de l'instance.



Règle de choix : cache pur → `allkeys-lru` ; base de données → `noeviction` ; usage mixte → `volatile-lru` avec TTL sur toutes les clés de cache. **Piège** : les clés de cache sans TTL deviennent invictables en mode `volatile-all`.

LRU Approximé par Échantillonnage

Redis n'implémente pas un LRU exact (trop coûteux en mémoire et CPU). Il utilise un LRU probabiliste par échantillonnage, suffisamment précis pour la quasi-totalité des cas d'usage.

 The picture can't be displayed.

Fonctionnement

À chaque besoin d'éviction, Redis échantillonne `maxmemory-samples` clés aléatoires (défaut : 5) et évince la moins récemment utilisée parmi cet échantillon.

 The picture can't be displayed.

Tuning

Augmenter `maxmemory-samples` à 10 rapproche le comportement du LRU exact au prix d'un léger overhead CPU. Valeur 5 est un bon compromis pour la production.

 The picture can't be displayed.

LFU (Redis 4+)

La politique LFU maintient un compteur de fréquence logarithmique par clé, décroissant dans le temps. Plus précise que LRU pour les accès non-uniformes.

Réplication Master-Replica

Redis supporte la réplication asynchrone un-vers-plusieurs. Un master peut avoir plusieurs replicas, chacun pouvant avoir ses propres replicas (cascade).



Connexion PSYNC

Sync complète

Sync partielle

Lecture seule



La réplication n'apporte pas le failover automatique. En cas de panne du master, une intervention manuelle est nécessaire — ou l'utilisation de **Redis Sentinel** (Module 5).

Réplication : Points Clés de Configuration

replica-read-only yes

Valeur par défaut et recommandée. Les replicas refusent les écritures directes, garantissant la cohérence avec le master.

Cohérence éventuelle

La réplication étant asynchrone, un client lisant sur un replica peut obtenir des données légèrement en retard. Utiliser `WAIT numreplicas timeout` pour une cohérence plus forte.

Backlog PSYNC

`repl-backlog-size` (défaut 1MB) détermine la taille du buffer de réplication. Si un replica se déconnecte trop longtemps, une resync complète (RDB) est nécessaire.

Lecture sur replicas

Distribuer les lectures sur les replicas soulage le master. Adapter l'application pour tolérer la cohérence éventuelle ou utiliser des données non critiques.

Synthèse du Module

Les concepts fondamentaux à retenir avant le QCM.

The picture can't be displayed.

RESP & Réseau

5 types primitifs, séquence requête/réponse. RESP3 ajoute les types riches et le client-side caching via push messages.

The picture can't be displayed.

Durabilité

RDB = snapshots compacts, AOF = journal `everysec` recommandé. Mode mixte = meilleur des deux mondes. `aof-use-rdb-preamble yes`.

The picture can't be displayed.

Mono-thread & Event Loop

Atomicité garantie par commande, `epoll/kqueue` pour le multiplexage. I/O threads depuis Redis 6 pour la couche réseau uniquement.

The picture can't be displayed.

Mémoire & Réplication

`maxmemory` à 60–75% RAM, politique selon le rôle. Réplication asynchrone RDB+backlog, `WAIT` pour cohérence forte. Sentinel pour failover automatique.

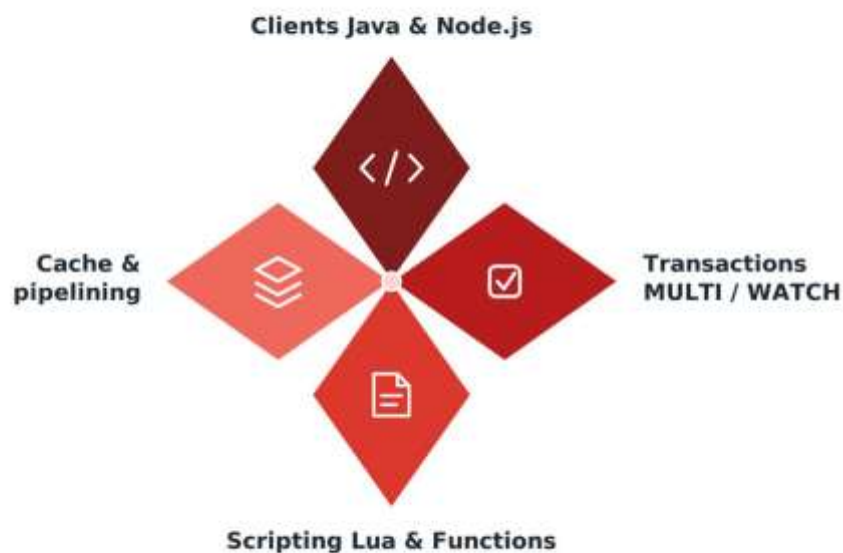
Ressources

Pour aller plus loin

- Documentation officielle : `redis.io/docs`
- `redis.conf` commenté — référence complète des paramètres
- Article : *Redis persistence demystified* (Antirez, 2012)
- Module 4 : Structures de données avancées
- Module 5 : Redis Sentinel & Cluster

Prochain module

Structures avancées : Streams, HyperLogLog, Modules Redis et cas d'usage distribués.



Module 4 : Développer avec Redis

De la connexion à la production : clients, cache, transactions, scripting Lua et pipelining. Ce module couvre les patterns essentiels pour intégrer Redis dans vos applications Java et Node.js avec robustesse et performance.

MODULE 4

Panorama des clients Redis par langage

Redis dispose d'un écosystème riche de clients officiels et communautaires. Chaque langage possède ses bibliothèques de référence, avec des niveaux de maturité et des paradigmes différents.



Java

Jedis, Lettuce, Redisson — sync, async, réactif



Go

go-redis — performant, idiomatique, supporte les clusters



Node.js

node-redis v4+ (officiel), ioredis — event-driven, async/await



C# / .NET

StackExchange.Redis — multiplexing natif, très utilisé en entreprise



Python

redis-py (officiel) — sync et asyncio avec **aioredis**

Jedis vs Lettuce vs Redisson

Trois clients Java majeurs, trois philosophies. Le choix dépend de votre modèle de concurrence et de vos besoins en fonctionnalités avancées.

Critère	Jedis	Lettuce	Redisson
Modèle	Synchrone	Sync / Async / Réactif	Sync / Async / Réactif
Thread-safe	(pool obligatoire)	(connexion partagée)	
Abstraction	Bas niveau	Bas niveau	Haut niveau (verrous, maps...)
Intégration Spring	Spring Data	Spring Data (défaut)	Spring Data

⚠ Jedis n'est pas thread-safe : n'utilisez jamais une instance brute dans un contexte multi-threadé sans pool.

Jedis : Pool de connexions et patterns essentiels

JedisPool — usage correct

```
JedisPoolConfig cfg = new JedisPoolConfig();
cfg.setMaxTotal(20);
cfg.setMaxIdle(5);
cfg.setMinIdle(2);

try (JedisPool pool = new JedisPool(cfg,
    "localhost", 6379)) {

    try (Jedis jedis = pool.getResource()) {
        // SET NX PX (atomic, TTL 30s)
        SetParams params = SetParams.setParams()
            .nx().px(30_000);
        String res = jedis.set("lock:order:42",
            "token-uuid", params);
        if ("OK".equals(res)) {
            // verrou acquis
        }
    } // auto-retour dans le pool
}
```

Bonnes pratiques

1 try-with-resources

Garantit le retour de la connexion dans le pool, même en cas d'exception.

2 SetParams nx().px()

SET atomique : crée la clé *seulement si elle n'existe pas*, avec TTL en ms.
Idéal pour les verrous distribués.

3 Taille du pool

Calculez : `maxTotal` $\approx$ threads applicatifs $\times$ temps moyen d'une commande Redis.

Spring Data Redis : RedisTemplate & Cache

Configuration & RedisTemplate

```
@Configuration
public class RedisConfig {
    @Bean
    public RedisTemplate<String, Object> redisTemplate(
        RedisConnectionFactory factory) {
        RedisTemplate<String, Object> tpl =
            new RedisTemplate<>();
        tpl.setConnectionFactory(factory);
        // Sérialisation JSON
        tpl.setDefaultSerializer(
            new GenericJackson2JsonRedisSerializer());
        tpl.setKeySerializer(
            new StringRedisSerializer());
        return tpl;
    }
}

// Utilisation
redisTemplate.opsForValue()
    .set("user:42", user, 1, TimeUnit.HOURS);
```

@Cacheable — cache déclaratif

```
@Service
@CacheConfig(cacheNames = "users")
public class UserService {

    @Cacheable(key = "#id")
    public User findById(Long id) {
        return userRepository.findById(id)
            .orElseThrow();
    }

    @CacheEvict(key = "#user.id")
    public User save(User user) {
        return userRepository.save(user);
    }
}
```

☐ Spring gère automatiquement la mise en cache et l'éviction — aucun code Redis explicite requis dans le service.

node-redis v4+ : Le client officiel

Initialisation robuste — singleton

```
import { createClient } from 'redis';

const client = createClient({
  url: 'redis://localhost:6379',
  socket: {
    reconnectStrategy: (retries) => {
      if (retries > 10) {
        return new Error('Max retries atteint');
      }
      // backoff exponentiel plafonné à 3s
      return Math.min(retries * 100, 3000);
    }
  }
});

// OBLIGATOIRE : écouteur d'erreur
client.on('error', (err) =>
  console.error('Redis Client Error:', err));

await client.connect();
export default client; // singleton
```

Règles d'or Node.js

 The picture can't be displayed.

Écouteur 'error' obligatoire

Sans lui, une erreur non gérée fait planter le process Node.js entier.

 The picture can't be displayed.

Singleton

Un seul client partagé dans toute l'app — la reconnexion est gérée automatiquement.

 The picture can't be displayed.

quit() vs disconnect()

`quit()` attend la fin des commandes en cours (propre). `disconnect()` coupe immédiatement.

Node.js : Types, scan et commandes bloquantes

Piège #1 — Redis ne connaît que des chaînes

scanIterator — itération sans blocage

```
// Ne jamais utiliser KEYS * en prod !
for await (const key of client.scanIterator({
  MATCH: 'user:* ',
  COUNT: 100
})) {
  console.log(key);
}
```

Piège #2 — Commandes bloquantes



Un client bloqué ne peut plus traiter d'autres commandes. Toujours utiliser `client.duplicate()` pour BLPOP, SUBSCRIBE, etc.

PATTERN

Module de cache réutilisable : Cache-Aside

Le pattern cache-aside est le plus répandu : l'application lit d'abord dans Redis, puis en base uniquement en cas de miss. Un incident Redis doit provoquer une dégradation gracieuse, jamais une panne.

Implémentation robuste du cache-aside

Node.js — module réutilisable

```
async function cacheAside(key, ttlMs, fetcher) {  
  try {  
    const cached = await client.get(key);  
    if (cached !== null)  
      return JSON.parse(cached);  
  } catch (err) {  
    // Redis indisponible → continuer sans cache  
    console.warn('Cache read failed:', err.message);  
  }  
  
  const data = await fetcher(); // appel DB/API  
  
  try {  
    // Jitter : ± 10% sur le TTL  
    const jitter = Math.random() * 0.2 - 0.1;  
    const finalTtl = Math.floor(ttlMs * (1 + jitter));  
    await client.set(key, JSON.stringify(data),  
      { PX: finalTtl });  
  } catch (err) {  
    console.warn('Cache write failed:', err.message);  
  }  
  return data;  
}
```

Principes de résilience

try/catch systématique

Chaque accès Redis est protégé.
Une erreur Redis = latence accrue,
pas une panne applicative.

Dégradation gracieuse

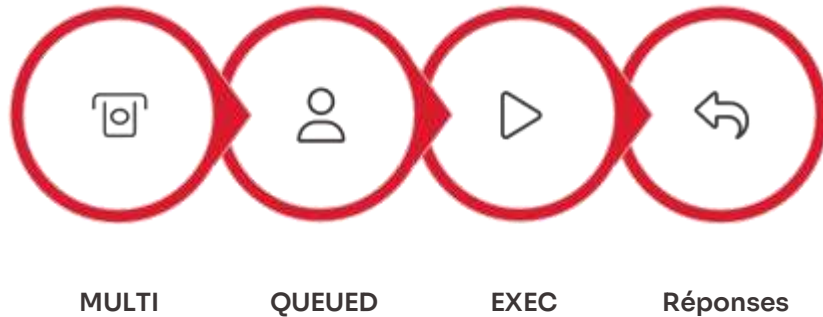
L'app continue de fonctionner sans
cache, en allant directement en
base.

Jitter sur le TTL

Évite le *cache stampede* : les clés n'expirent pas toutes simultanément sous forte charge.

MULTI / EXEC / DISCARD : Transactions Redis

Séquence MULTI/EXEC



Exemple Java avec Jedis

```
try (Jedis jedis = pool.getResource()) {
    Transaction tx = jedis.multi();
    try {
        tx.decrBy("stock:item:7", 1);
        tx.incrBy("sales:total", price);
        tx.lpush("orders:log", orderId);
        List<Object> results = tx.exec();
        // results[0], [1], [2] = résultats
    } catch (Exception e) {
        tx.discard();
        throw e;
    }
}
```

⚠ Pas de rollback ! Si une commande échoue à l'exécution (erreur de type, par ex.), les autres continuent. C'est une différence fondamentale avec SQL.

WATCH : Verrouillage optimiste lire-décider-écrire

Pattern WATCH en Node.js

```
async function transferWithWatch(from, to, amount) {
  let retries = 3;
  while (retries-- > 0) {
    // 1. Observer la clé
    await client.watch(from);

    const balance =
      Number(await client.get(from));
    if (balance < amount) {
      await client.unwatch();
      throw new Error('Solde insuffisant');
    }

    // 2. Transaction conditionnelle
    const results = await client
      .multi()
      .decrBy(from, amount)
      .incrBy(to, amount)
      .exec(); // null si clé modifiée !

    if (results !== null) return results;

    // 3. Backoff avant retry
    await sleep(50 * (4 - retries));
  }
  throw new Error('Transaction échouée après retries');
}
```

Fonctionnement de WATCH

01

WATCH key

Redis surveille la clé. Toute modification par un autre client avant EXEC annule la transaction.

02

Lire & décider

Lire la valeur actuelle et calculer les nouvelles valeurs à écrire.

03

MULTI/EXEC

Si EXEC retourne `null`, la clé a été modifiée : réessayer avec backoff.

04

UNWATCH

Annuler la surveillance si on décide de ne pas exécuter la transaction.

Scripts Lua : Atomicité côté serveur

Les scripts Lua s'exécutent **atomiquement** sur le serveur Redis : aucune autre commande ne peut s'intercaler. C'est la solution aux problèmes de race condition que ni les transactions MULTI/EXEC ni le pipeline ne résolvent seuls.

EVAL, EVALSHA, SCRIPT LOAD

```
-- Rate limiter atomique
-- KEYS[1] = clé compteur, ARGV[1] = limite,
-- ARGV[2] = TTL en secondes
local current = redis.call('INCR', KEYS[1])
if current == 1 then
    redis.call('EXPIRE', KEYS[1], tonumber(ARGV[2]))
end
if current > tonumber(ARGV[1]) then
    return 0 -- limite dépassée
end
return 1 -- requête autorisée
```

```
// Exécution depuis Node.js
const allowed = await client.eval(luaScript, {
  keys: ['ratelimit:user:42'],
  arguments: ['100', '60']
});
```

Libération sûre d'un verrou

```
-- Vérifie le token AVANT de supprimer
-- Évite de libérer le verrou d'un autre process
local val = redis.call('GET', KEYS[1])
if val == ARGV[1] then
    return redis.call('DEL', KEYS[1])
else
    return 0
end
```

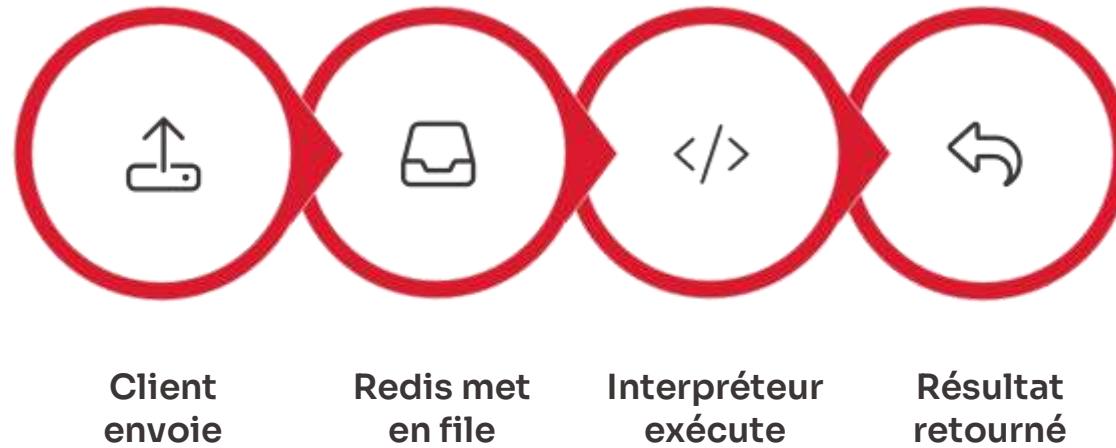
KEYS[] & ARGV[]

Toujours déclarer les clés dans `KEYS[]`
— obligatoire en mode cluster pour le routage.

Scripts courts & déterministes

Les scripts bloquent Redis. Éviter les boucles longues. Respecter `lua-time-limit` (5s par défaut).

Exécution d'un script Lua côté serveur



EVALSHA & SCRIPT LOAD — optimisation réseau

```
// Charger une fois, exécuter par hash SHA1
const sha = await client.scriptLoad(luaScript);
// Ultérieurement (pas de retransmission du script)
const result = await client.evalSha(sha, {
  keys: ['mykey'],
  arguments: ['arg1']
});
```

Redis Functions (Redis 7+)

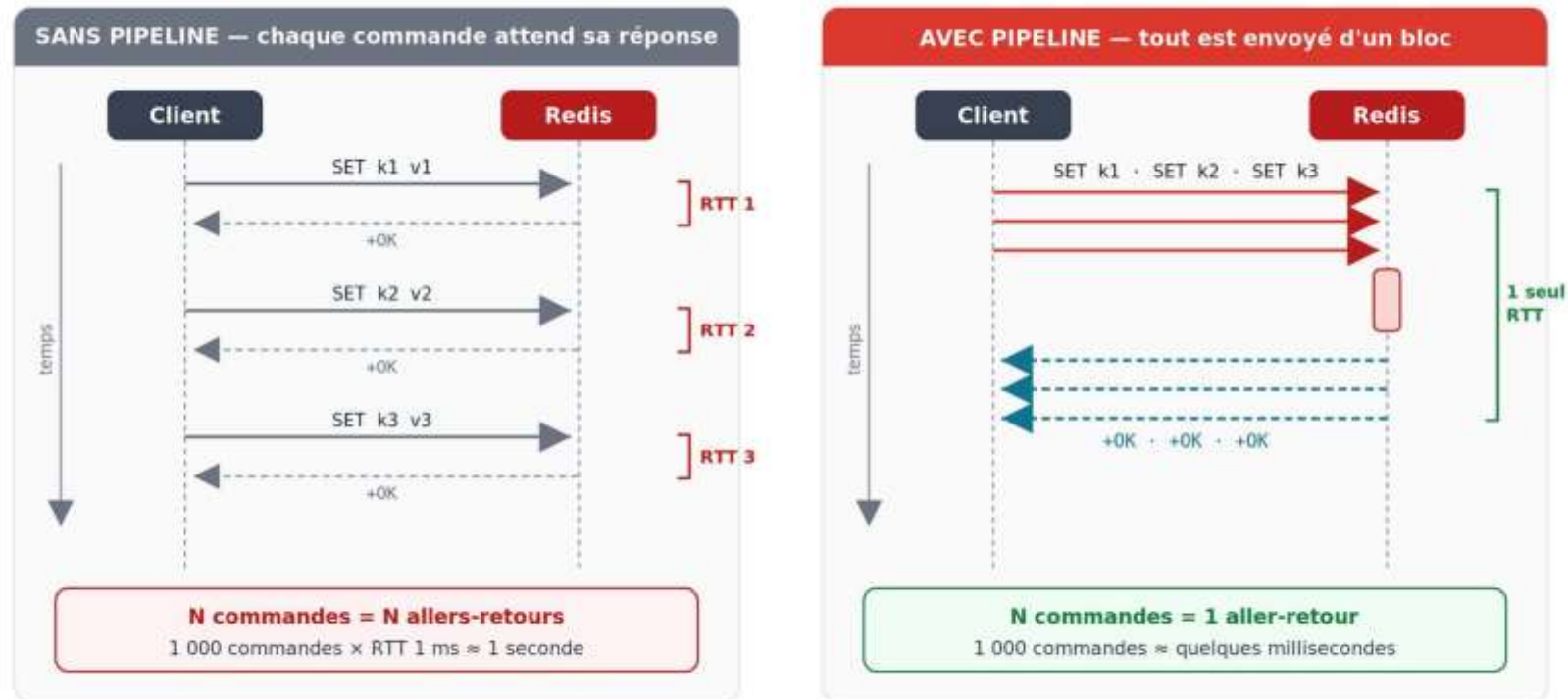
```
#!/lua name=mylib
local function my_func(keys, args)
  return redis.call('GET', keys[1])
end
redis.register_function('my_func', my_func)
```

Les Redis Functions sont persistées sur disque et répliquées automatiquement — elles remplacent le cache de scripts EVALSHA pour les déploiements en production.

Pipelining : optimisation réseau radicale

Sans pipeline, chaque commande attend sa réponse avant d'envoyer la suivante. Avec le pipeline, toutes les commandes sont envoyées d'un bloc, réduisant drastiquement la latence réseau cumulée.

Pipelining : optimisation réseau radicale



Le pipelining est une optimisation réseau, pas une transaction — procéder par lots de 1 000 à 10 000 commandes

MSET / MGET · client.multi() · Promise.all() — déjà pipelinés

Java — Jedis pipeline

```
try (Jedis jedis = pool.getResource()) {
    Pipeline pipe = jedis.pipelined();
    for (int i = 0; i < 1000; i++) {
        pipe.set("key:" + i, "value:" + i);
    }
    // Envoi groupé + réception des réponses
    List<Object> responses = pipe.syncAndReturnAll();
}
```

Node.js — Pipeline automatique

```
// Promise.all est déjà pipeliné par node-redis !
const [user, prefs, cart] = await Promise.all([
    client.get('user:42'),
    client.hGetAll('prefs:42'),
    client.lRange('cart:42', 0, -1)
]);

// Pipeline explicite si nécessaire
const pipeline = client.multi();
for (let i = 0; i < 1000; i++) {
    pipeline.set(`key:${i}`, `value:${i}`);
}
await pipeline.exec();
```

📌 Le pipelining est différent des transactions : les commandes ne sont pas atomiques. D'autres clients peuvent écrire entre deux lots. Recommandé : lots de 500 à 10 000 commandes.

Gestion des connexions : pool, timeouts et TLS

Configuration pool Java (Lettuce)

```
GenericObjectPoolConfig<StatefulRedisConnection>
    poolConfig = new GenericObjectPoolConfig<>();
poolConfig.setMaxTotal(20);
poolConfig.setMaxWait(Duration.ofMillis(500));

// TLS avec rediss://
RedisClient client = RedisClient.create(
    "rediss://redis.prod.example.com:6380");
client.setOptions(ClientOptions.builder()
    .socketOptions(SocketOptions.builder()
        .connectTimeout(Duration.ofSeconds(3))
        .build())
    .build());
```

Configuration pool Node.js

```
const client = createClient({
  url: 'rediss://redis.prod.example.com:6380',
  socket: {
    connectTimeout: 3000,      // 3s
    commandTimeout: 500,      // 500ms
    tls: true,
    reconnectStrategy: (retries) =>
      Math.min(retries * 150, 5000)
  }
});
```

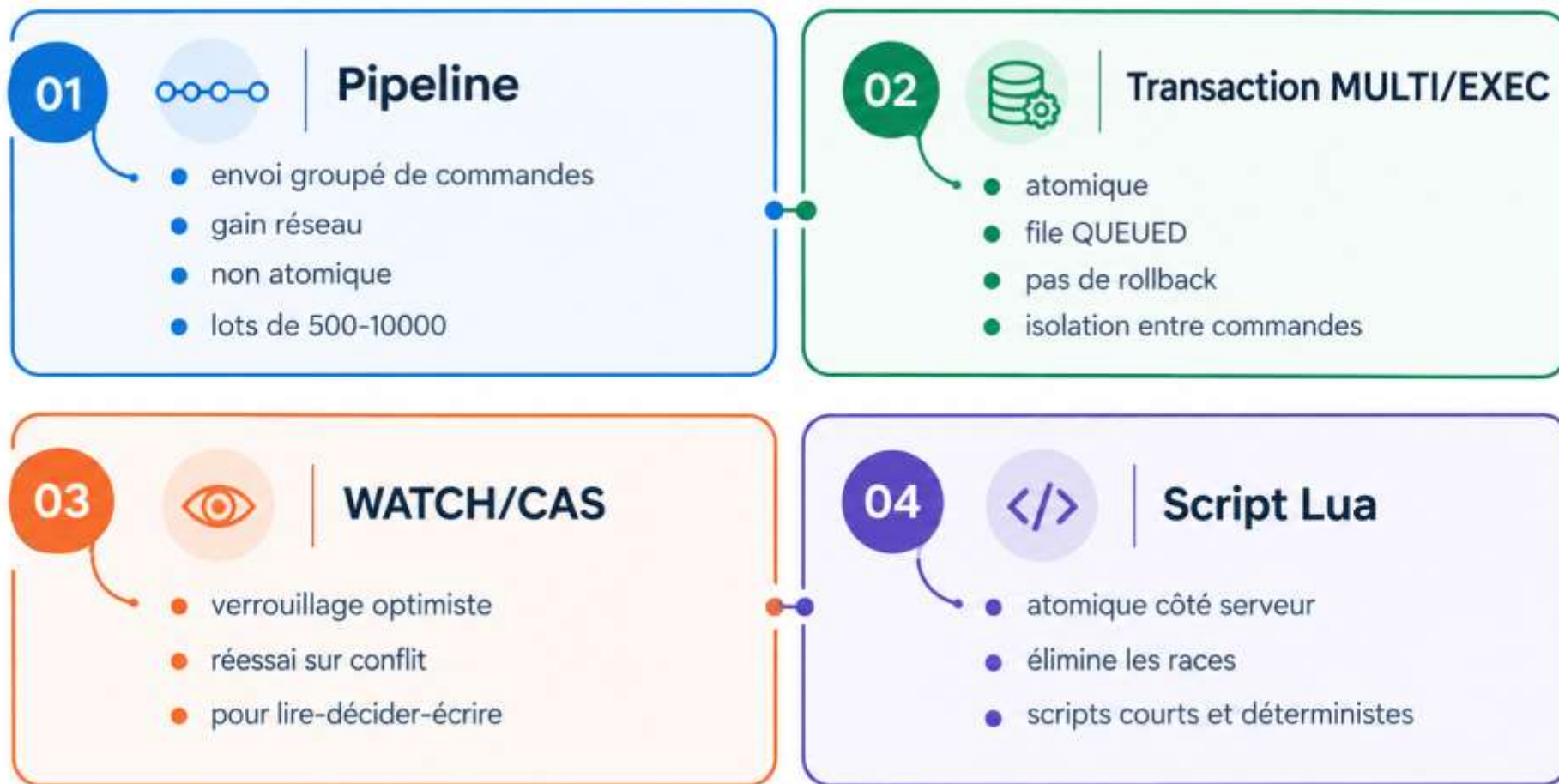
rediss://

Préfixe pour activer TLS —
indispensable hors réseau privé.

commandTimeout

Limite le temps d'attente d'une
réponse Redis, évite les
blocages silencieux.

Récapitulatif : Quand utiliser quoi ?



TP Java : Jedis → Spring Data Redis

Étape 1 — Jedis brut

```
// pom.xml : redis.clients:jedis:5.x
public class RateLimiterService {
    private final JedisPool pool;
    private static final String LUA =
        "local c = redis.call('INCR', KEYS[1])\n" +
        "if c == 1 then\n" +
        "  redis.call('EXPIRE', KEYS[1], ARGV[2])\n" +
        "end\n" +
        "return c <= tonumber(ARGV[1]) and 1 or 0";

    public boolean isAllowed(String userId) {
        try (Jedis j = pool.getResource()) {
            Object r = j.eval(LUA,
                List.of("r1:" + userId),
                List.of("100", "60"));
            return Long.parseLong(r.toString()) == 1L;
        }
    }
}
```

Étape 2 — Spring Data Redis

```
// application.yml
spring:
  data:
    redis:
      host: localhost
      port: 6379
  cache:
    type: redis
    redis:
      time-to-live: 3600000 # 1h en ms

// Service
@Service
public class UserService {
    @Autowired RedisTemplate<String, User> tpl;

    @Cacheable(value="users", key="#id")
    public User getUser(Long id) {
        return db.findById(id).orElseThrow();
    }

    public void watchedTransfer(
        String from, String to, long amount) {
        // Implémenter avec SessionCallback + WATCH
    }
}
```

DEMO Node.js : Structure du projet

Architecture recommandée

```
src/
├── redis/
│   ├── client.js      # singleton + reconnect
│   └── cache.js       # module cache-aside
├── services/
│   ├── userService.js # cache utilisateurs
│   ├── counter.js     # INCR + pipeline
│   ├── transfer.js    # WATCH/EXEC
│   └── ratelimiter.js  # script Lua
├── routes/
│   └── api.js
└── index.js
```

Tests de charge suggérés

```
# Installer autocannon
npx autocannon -c 50 -d 10 \
  http://localhost:3000/api/view/home

# Vérifier le compteur
redis-cli GET pageviews:home
```

transfer.js — WATCH complet

```
import client from '../redis/client.js';

export async function transfer(from, to, amt) {
  for (let attempt = 0; attempt < 5; attempt++) {
    await client.watch(from);
    const bal = Number(await client.get(from));

    if (bal < amt) {
      await client.unwatch();
      throw new Error('Solde insuffisant');
    }

    const res = await client.multi()
      .decrBy(from, amt)
      .incrBy(to, amt)
      .exec();

    if (res !== null) {
      console.log(`Transfert OK, tentative ${attempt+1}`);
      return res;
    }
    // Backoff exponentiel
    await new Promise(r =>
      setTimeout(r, 2 ** attempt * 50));
  }
  throw new Error('Transaction échouée');
}
```

Synthèse du Module 4

Clients Java

Jedis (pool obligatoire), Lettuce (async/réactif), Spring Data Redis (@Cacheable, RedisTemplate)

Node.js

Singleton + écouteur 'error' + reconnectStrategy. Types stricts, scanIterator, client.duplicate() pour le blocage

Cache & Résilience

Cache-aside avec try/catch, dégradation gracieuse, jitter TTL pour éviter le stampede

Transactions & Lua

MULTI/EXEC sans rollback, WATCH pour l'optimisme, Lua pour l'atomicité fine, Redis Functions pour la persistance

Performance

Pipeline : 1000 commandes en quelques ms. Lots de 500-10 000. Promise.all déjà pipeliné côté Node.js

Ressources & Prochaines étapes

Documentation officielle



redis.io/docs

Référence complète des commandes, guides par langage, exemples interactifs



github.com/redis/node-redis

Documentation node-redis v4+, exemples, changelog



github.com/redis/jedis

Wiki Jedis, javadoc, patterns recommandés



spring.io/projects/spring-data-redis

Reference documentation Spring Data Redis, auto-configuration

Module 5 — Aperçu

01

Redis en production

Réplication, Sentinel, Redis Cluster — haute disponibilité et partitionnement horizontal

02

Monitoring & Observabilité

INFO, MONITOR, Latency Tracking, intégration Prometheus/Grafana

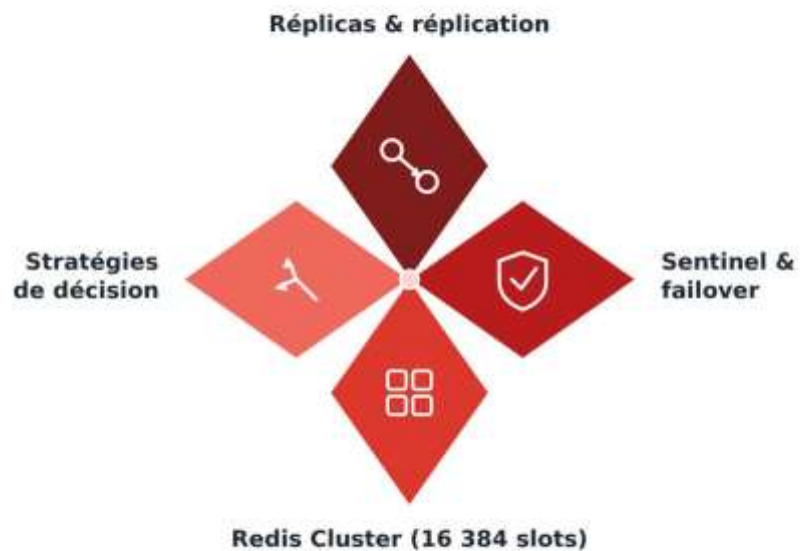
03

Sécurité

ACL, AUTH, TLS mutuel, réseau privé — sécuriser Redis en entreprise



Le TP est à rendre avant le prochain cours. Un corrigé commenté sera mis à disposition après la date limite.



Module 5 : Haute disponibilité et clustering Redis

Garantir la continuité de service dans les architectures Redis critiques — réplicas, Sentinel, Cluster, et stratégies de décision pour les architectes et ingénieurs de la donnée.

MODULE 5

HAUTE DISPONIBILITÉ

Enjeux : RPO, RTO et types de pannes

Définitions fondamentales

RPO (Recovery Point Objective) : quelle quantité de données peut-on tolérer de perdre ? Exprimé en secondes ou minutes de données non persistées.

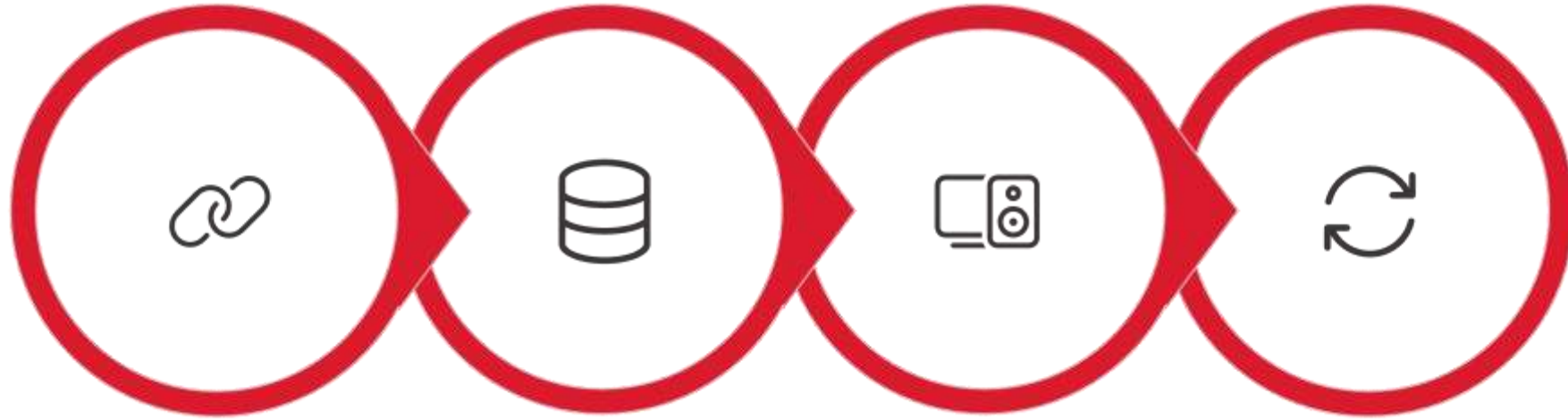
RTO (Recovery Time Objective) : combien de temps le service peut-il rester indisponible avant impact métier critique ?

Ces deux métriques guident tous les choix architecturaux de haute disponibilité.

Taxonomie des pannes

- Panne processus : crash du processus Redis, redémarrage possible sans perte réseau.
- Panne machine : défaillance matérielle, OS ou alimentation — nœud indisponible pour une durée indéterminée.
- Panne réseau : partition réseau, isolant des nœuds sans les arrêter (scénario split-brain).
- Panne datacenter : sinistre géographique, exige une architecture multi-site.

Réplicas Redis : cycle de vie et propagation



Connexion

FULLRESYNC

Propagation

Resynchronisation

La réplication asynchrone garantit de hautes performances en écriture mais implique un lag potentiel entre master et replica. Le paramètre `repl-backlog-size` dimensionne le tampon circulaire permettant la resynchronisation partielle (PSYNC) après une déconnexion brève, évitant une re-synchronisation totale coûteuse.

Réplication : comportements clés à maîtriser

 The picture can't be displayed.

Promotion manuelle

La commande `REPLICAOF NO ONE` détache un replica et le promeut en master indépendant.

Opération manuelle, sans coordination automatique — à utiliser prudemment en production.

 The picture can't be displayed.

Gestion des expirations

Les clés TTL sont gérées **exclusivement par le master**.

Lorsqu'une clé expire, le master envoie une commande `DEL` explicite aux replicas pour garantir la cohérence — les replicas ne suppriment jamais de clé de façon autonome.

 The picture can't be displayed.

Resynchronisation totale

Si le backlog est saturé ou si le replica s'est reconnecté trop tard, une **FULLRESYNC** est déclenchée : le master génère un snapshot RDB complet et le transfère — opération I/O intensive à surveiller.

Redis Sentinel : les 4 rôles fondamentaux



Monitoring

Vérifie en continu la disponibilité du master et des replicas via des pings périodiques.



Notification

Alerte les opérateurs et systèmes externes via API ou scripts lors de changements d'état.



Failover automatique

Promeut un replica en master sans intervention humaine en cas de défaillance confirmée.



Service Discovery

Fournit aux clients l'adresse courante du master — abstraction du changement de topologie.

Configuration Sentinel : `sentinel.conf`

Directives essentielles

`sentinel monitor`

Déclare le master à surveiller : nom logique, IP, port et quorum (nombre minimal de sentinelles devant voter pour confirmer une panne).

`down-after-milliseconds`

Délai en ms sans réponse avant qu'un nœud soit déclaré `SDOWN` (subjectivement indisponible) par un sentinel.

`parallel-syncs`

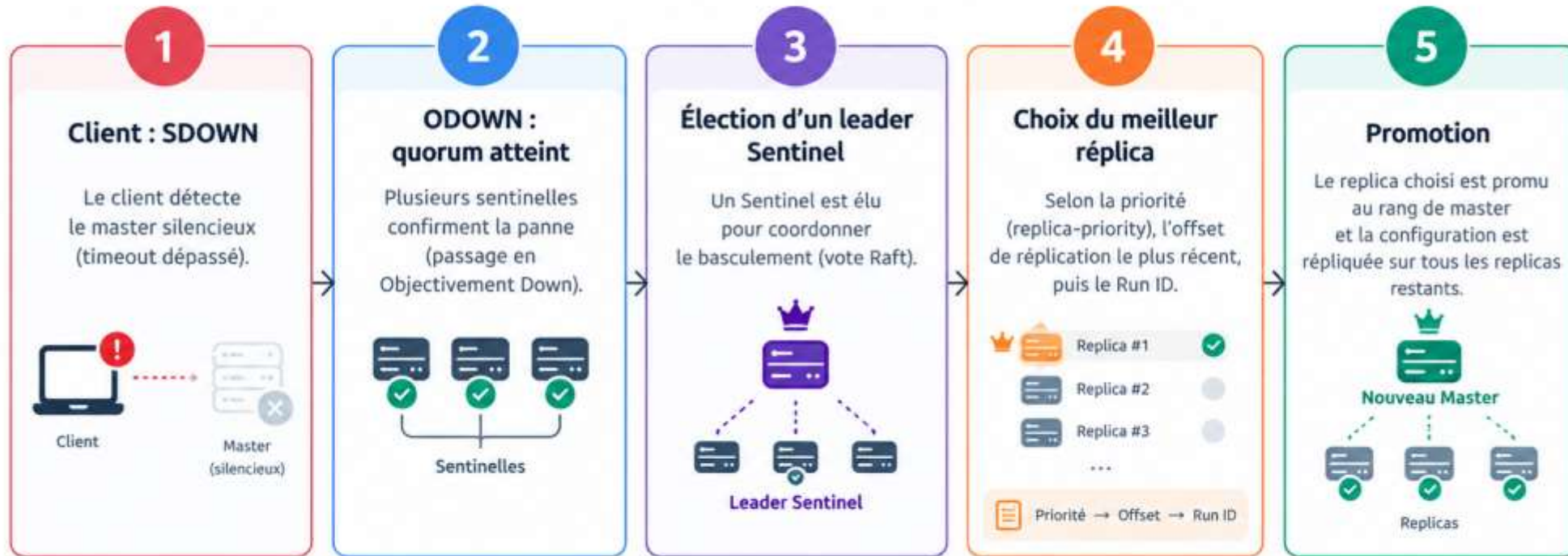
Nombre de replicas autorisés à se resynchroniser simultanément avec le nouveau master post-failover. Limiter à 1 pour préserver la disponibilité en lecture.

Exemple minimal

```
sentinel monitor mymaster 127.0.0.1 6379 2
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 60000
sentinel parallel-syncs mymaster 1
```

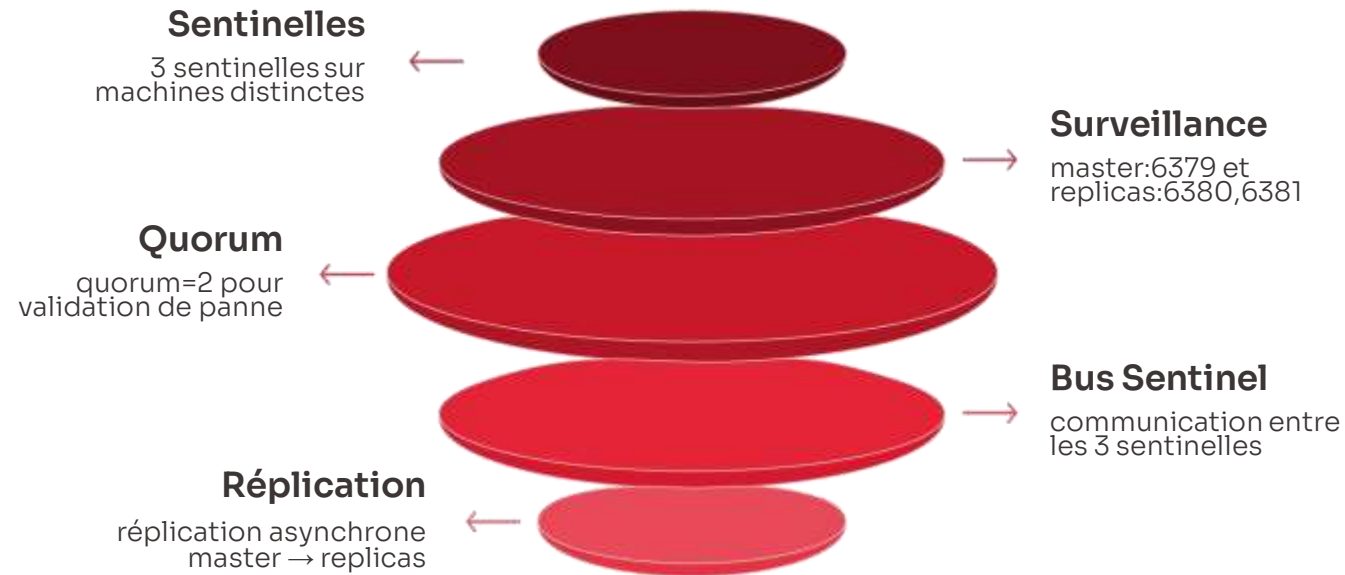
❏ Le quorum doit être inférieur ou égal au nombre total de sentinelles. Avec 3 sentinelles, un quorum de 2 est la valeur recommandée.

Scénario de bascule Sentinel en 5 étapes



L'ensemble du processus est automatique et prend typiquement **quelques secondes** selon les valeurs de `down-after-milliseconds` et `failover-timeout`. Le leader sentinel orchestre seul la promotion et la reconfiguration.

Architecture Sentinel : topologie recommandée



Règle des 3 sentinelles minimum

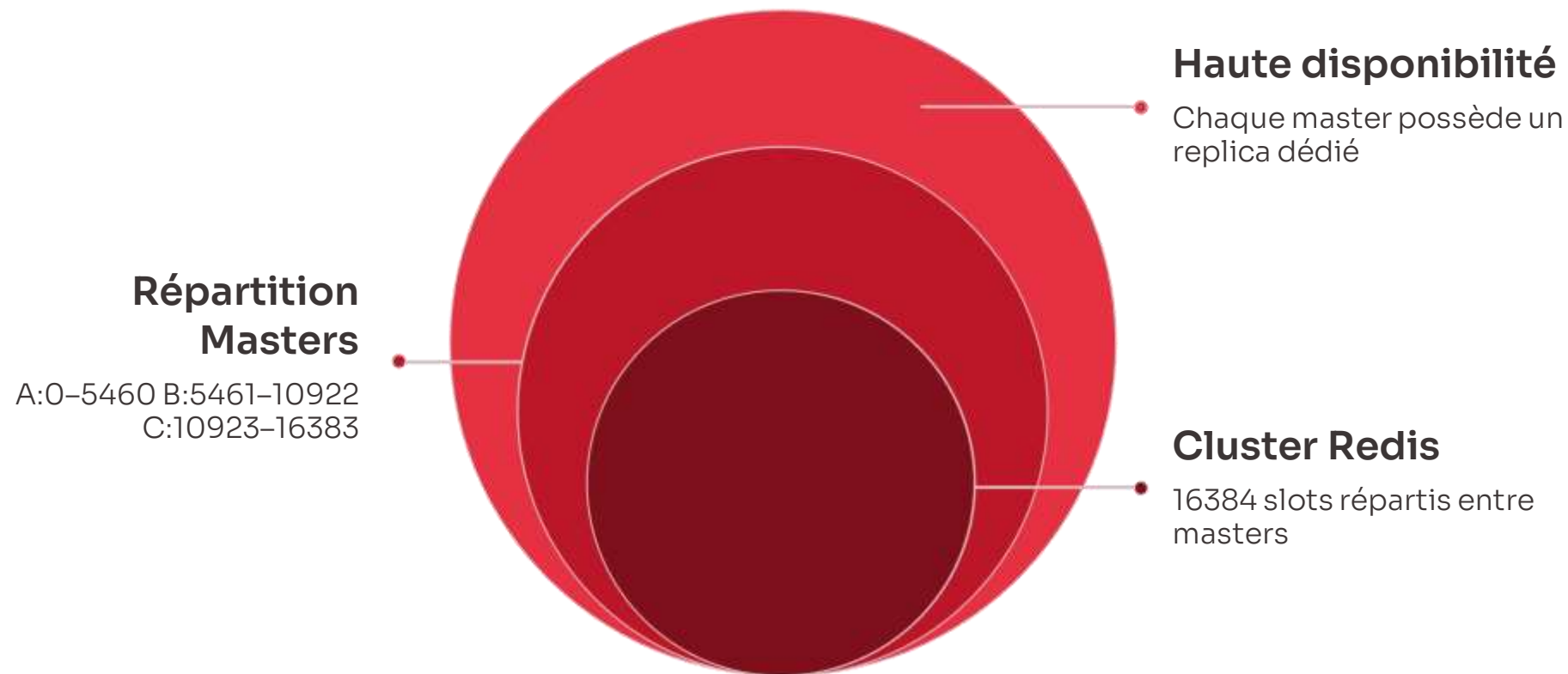
Trois instances sur des **machines physiques distinctes** garantissent qu'une panne unique ne compromet pas le quorum. Deux sentinelles ne suffisent pas : si l'une tombe avec le master, le quorum est impossible.

Connexion côté client

Le client se connecte aux **sentinelles**, jamais directement au master. Exemple avec `ioredis` :

```
new Redis({
  sentinels: [
    { host: 's1', port: 26379 },
    { host: 's2', port: 26379 },
    { host: 's3', port: 26379 }
  ],
  name: 'mymaster'
});
```

Redis Cluster : partitionnement par hash slots



Redis Cluster répartit l'espace de clés en 16 384 hash slots calculés par $\text{CRC16}(\text{key}) \bmod 16384$. Cette répartition fixe permet d'ajouter ou retirer des nœuds dynamiquement en déplaçant uniquement les slots concernés, sans indisponibilité globale.

Redirections MOVED et ASK

Moved — Redirection permanente

Le nœud contacté n'est pas responsable du slot demandé. Il retourne `Moved <slot> <ip:port>` indiquant le nœud définitif. Le client doit mettre à jour sa table de routage locale.

```
GET user:1042
-MOVED 7638 192.168.1.2:6379
```

ASK — Redirection temporaire

Pendant un resharding, un slot est en cours de migration. Le client reçoit `ASK` et doit envoyer `ASKING` avant sa commande sur le nœud destination — sans mettre à jour sa table, car la migration n'est pas terminée.

```
GET order:5510
-ASK 5510 192.168.1.3:6379
# Le client envoie ASKING puis réitère GET
```

Ces mécanismes permettent un resharding en ligne sans interruption de service.

Contrainte CROSSSLOT et hash tags

1

Le problème CROSSSLOT

Les opérations multi-clés (MGET, MSET), transactions (MULTI/EXEC) et scripts Lua exigent que toutes les clés appartiennent au même slot. Une erreur CROSSSLOT est retournée sinon.

2

Solution : les hash tags {}

En entourant une partie de la clé avec des accolades, seul ce fragment est haché. Ainsi {user:1042}:session et {user:1042}:cart atterrissent dans le même slot, permettant les opérations groupées.

3

Autres limites du Cluster

Base 0 uniquement — les bases multiples (SELECT) ne sont pas supportées. Pub/Sub : les messages sont diffusés à tous les nœuds (Redis 7 introduit SSUBSCRIBE/SPUBLISH pour un Pub/Sub shardé). Complexité opérationnelle accrue.

Resharding et gestion des nœuds en ligne

01

Ajout d'un nœud

Démarrer le nouveau nœud Redis, l'ajouter au cluster via `CLUSTER MEET`, puis redistribuer des slots avec `redis-cli --cluster reshard`.

02

Migration des slots

Les clés migrent slot par slot entre source et destination. Pendant la migration, les redirections ASK assurent la continuité. Aucune interruption globale.

03

Retrait d'un nœud

Vider d'abord tous ses slots vers d'autres nœuds via reshard, puis exécuter `CLUSTER FORGET` sur chaque membre du cluster pour l'exclure proprement.

Sentinel vs Cluster : tableau de choix

Critère	Redis Sentinel	Redis Cluster
Objectif principal	Haute disponibilité (failover automatique)	Scalabilité horizontale + HA
Partitionnement	Non — toutes les données sur un master	Oui — 16 384 hash slots sur N masters
Volume de données	Jusqu'à la RAM d'une machine (~centaines de Go)	Illimité (ajout de nœuds)
Multi-clés/ transactions	Supportées nativement	Limitées au même slot (hash tags requis)
Complexité opérationnelle	Modérée (3 sentinelles minimum)	Élevée (6 nœuds minimum recommandés)
Pub/Sub	Complet	Diffusé globalement (SSUBSCRIBE en Redis 7)
Bases multiples	Supportées (SELECT 0-15)	Non — base 0 uniquement

❏ Règle d'or : n'adopter Redis Cluster que si le volume de données ou le débit d'écriture dépassent la capacité d'une seule machine. Une instance moderne bien dimensionnée gère facilement des centaines de Go.

Prévention du split-brain

Le risque split-brain

Une partition réseau peut isoler le master actuel, qui continue à accepter des écritures pendant que Sentinel promeut un nouveau master. On se retrouve avec deux masters simultanés acceptant des données divergentes — les écritures sur l'ancien master seront perdues lors de la réconciliation.

Paramètres de protection

`min-replicas-to-write` `N` : le master refuse les écritures s'il dispose de moins de `N` replicas connectés et à jour. Rend l'ancien master inopérant lors d'une partition.

`cluster-require-full-coverage` `yes` : en mode Cluster, le cluster refuse les requêtes si certains slots ne sont pas couverts, évitant les réponses partielles trompeuses.

Ces deux paramètres ensemble réduisent drastiquement l'exposition au split-brain.

Tableau de décision architectural

Architecture	Quand l'adopter	Prérequis
Instance unique	Développement, cache non critique, données éphémères	Aucun — configuration minimale
Master + Replicas	Lecture intensive, sauvegarde, reporting — failover manuel accepté	2+ nœuds, réplication configurée
Sentinel	Production critique, failover automatique requis, données tenant sur une machine	3 sentinelles min., clients compatibles
Cluster	Volume > RAM d'un serveur, débit d'écriture dépassant une instance, sharding obligatoire	6 nœuds min. (3 masters + 3 replicas), refactoring applicatif
Service managé	Équipe ops réduite, cloud-native, SLA garanti par le fournisseur	Budget cloud, acceptation du vendor lock-in

DEMO : Cluster 6 nœuds et resharding

Création du cluster

```
redis-cli --cluster create \  
  127.0.0.1:7000 127.0.0.1:7001 \  
  127.0.0.1:7002 127.0.0.1:7003 \  
  127.0.0.1:7004 127.0.0.1:7005 \  
  --cluster-replicas 1
```

Redis distribue automatiquement les 16 384 slots sur les 3 masters et associe un replica à chacun.

Resharding et validation

```
# Ajouter un 4e master  
redis-cli --cluster add-node \  
  127.0.0.1:7006 127.0.0.1:7000  
  
# Resharding : déplacer 4096 slots  
redis-cli --cluster reshard \  
  127.0.0.1:7000
```

Vérifications attendues



Observer les redirections MOVED lors de l'accès à des clés migrées.



Tester une opération CROSSSLOT et la corriger avec les hash tags {}.



Valider la répartition finale avec `CLUSTER INFO` et `CLUSTER SLOTS`.

Synthèse : les règles d'or de la HA Redis



Répliquer systématiquement

Toute instance de production doit avoir au moins un replica. Sans réplication, une panne matérielle entraîne une perte de données irrémédiable.



Automatiser le failover

Sentinel est le minimum acceptable en production critique. Un failover manuel dépend d'une astreinte humaine — trop lent pour les applications à fort RTO.



Choisir la complexité juste

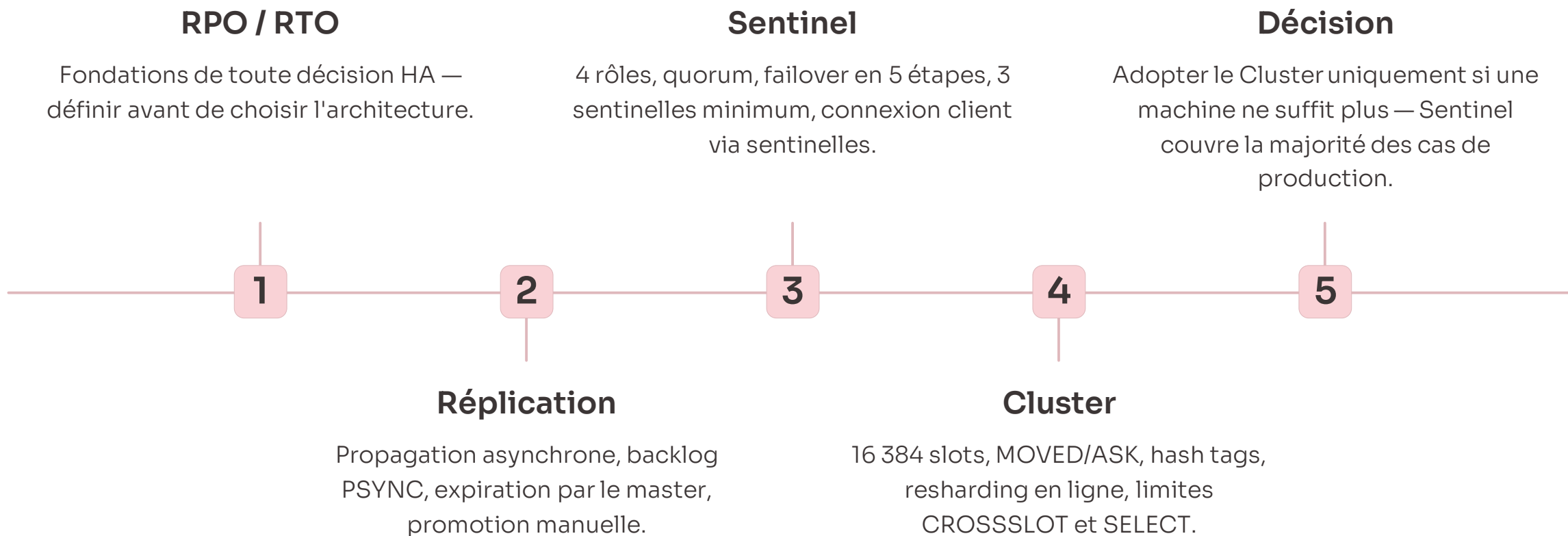
Le Cluster n'est pas la solution universelle. Une instance Redis moderne gère des centaines de Go. Mesurer avant de scaler horizontalement.



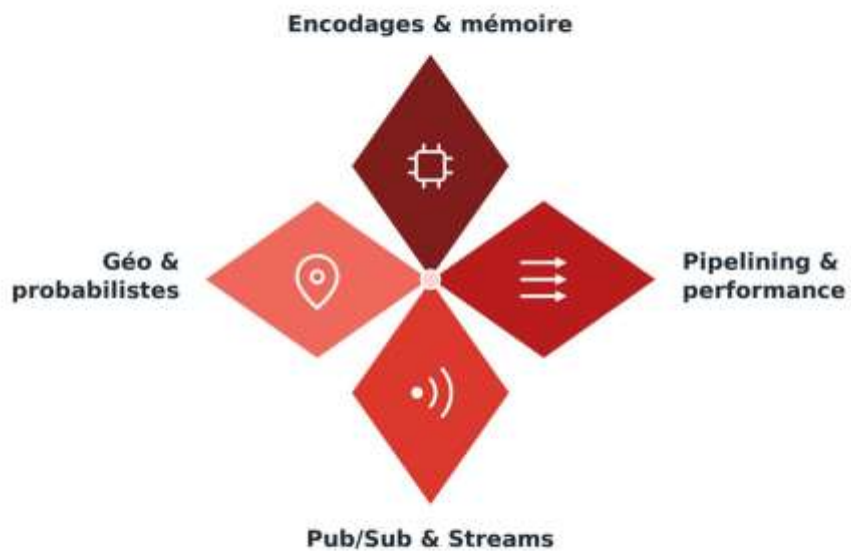
Protéger contre le split-brain

Configurer `min-replicas-to-write` sur tout master de production. Le split-brain silencieux est plus dangereux qu'une indisponibilité franche.

Récapitulatif du Module 5



📌 Prochain module : observabilité, métriques Redis et intégration Prometheus/Grafana pour monitorer ces architectures en production.



Module 6 : Optimisation et Structures Avancées

Encodages internes, pipelining, Pub/Sub, Streams, géolocalisation et structures probabilistes — maîtriser Redis au niveau expert pour concevoir des systèmes performants et économes en mémoire.

MODULE 6

AVANCÉ

Plan du Module

01

Expiration & Réplication

Lazy vs active, impact sur la mémoire et les réplicas

02

Pipelining & Encodages

Réduction des allers-retours, types internes, hash sharding

03

Pub/Sub & Streams

Fire-and-forget vs journal durable avec consumer groups

04

Structures spécialisées

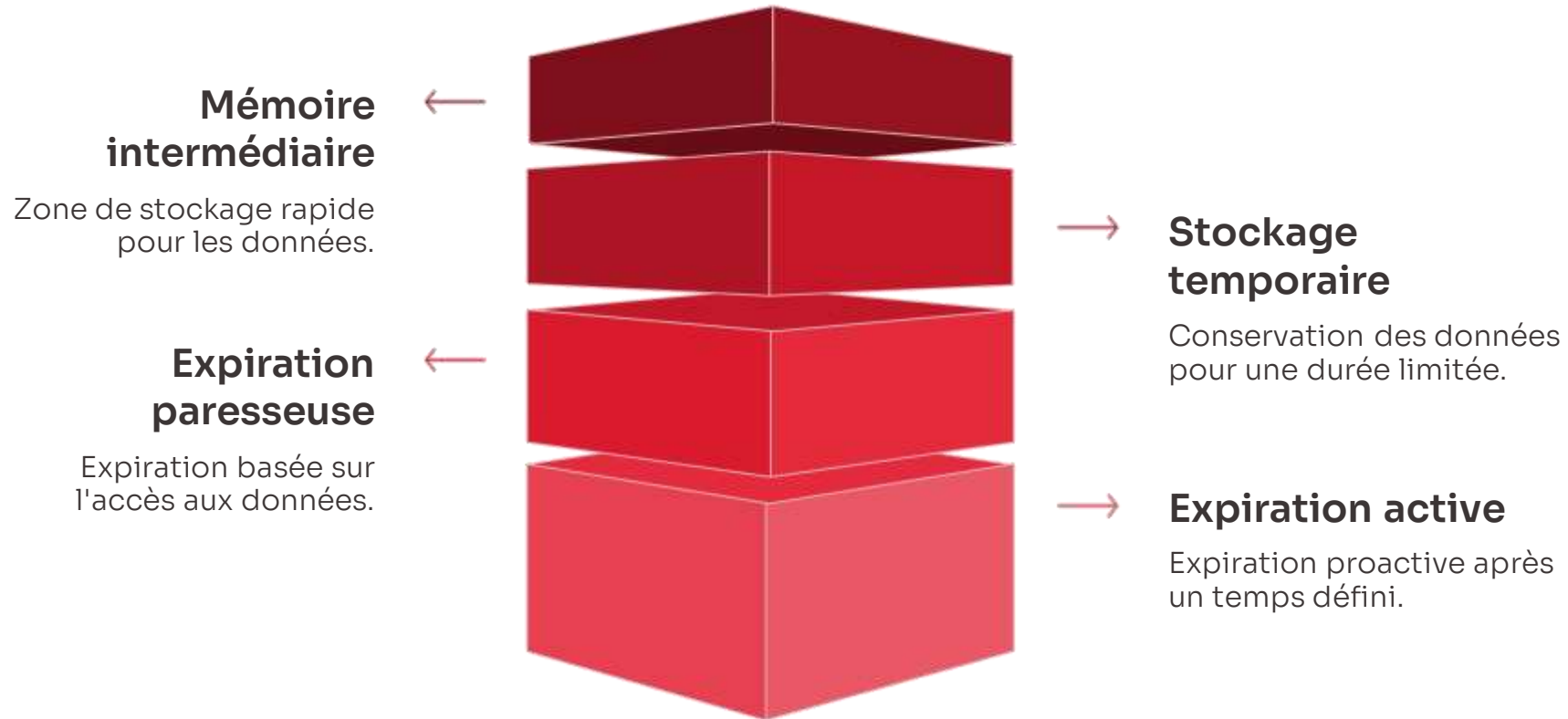
Géolocalisation, HyperLogLog, BitMap, anti-patterns

05

TP & Synthèse

Analytics complet + QCM de validation

Lazy vs Active Expiration



Une clé expirée **occupe toujours** la **mémoire** jusqu'à sa suppression effective. En réplication, la commande **DEL** est propagée aux réplicas uniquement lors de la suppression physique — un réplica peut donc temporairement retourner une clé logiquement expirée si le primaire ne l'a pas encore purgée.

Pipelining & Commandes Multi-Arguments

Sans Pipeline

Chaque commande attend la réponse du serveur avant d'envoyer la suivante. Latence réseau $\times$ N commandes = goulot d'étranglement majeur.

- SET k1 v1 → attente RTT
- SET k2 v2 → attente RTT
- SET k3 v3 → attente RTT

Avec Pipeline / Multi-arg

Regrouper les commandes en un seul envoi réseau réduit drastiquement les round-trips. Préférer les commandes variadic natives.

- MSET k1 v1 k2 v2 k3 v3
- SADD myset a b c d e
- LPUSH mylist x y z

📌 Le pipeline est côté client — les commandes ne sont pas atomiques entre elles. Pour l'atomicité, utiliser MULTI/EXEC ou un script Lua.

Types Logiques vs Types Physiques

Redis optimise automatiquement l'encodage interne selon la taille et le contenu de la structure. Utilisez `OBJECT ENCODING` clé pour inspecter l'encodage réel.

Type logique	Encodage compact	Encodage étendu	Seuil de conversion
String	int / embstr	raw	> 44 octets
Hash	listpack	hashtable	> 128 champs ou valeur > 64o
List	listpack	quicklist	> 128 éléments ou > 64o
Set	intset / listpack	hashtable	> 512 entiers ou élément non-int
Sorted Set	listpack	skiplist	> 128 membres ou > 64o

Hash Sharding : Économie Mémoire Spectaculaire

Le problème : 1 million de petits objets

Stocker 1 million de clés String individuelles génère une surcharge massive : chaque clé ajoute ~50–70 octets de métadonnées Redis (dictionnaire, TTL, type, encodage). Le coût réel est souvent 3 à 5× la taille des données utiles.

La solution : regrouper en 10 000 Hashes

En regroupant 100 entrées par Hash (`listpack` actif), on élimine la surcharge par clé. L'exemple canonique : `HSET user:[id/100]` `[id%100] valeur`. Résultat : économie de 60 à 80 % de mémoire RAM.

1M

Clés String

≈ 150 Mo RAM

10K

Hashes listpack

≈ 30 Mo RAM

Patterns de Modélisation Avancée

 The picture can't be displayed.

1

Index Secondaires

Utiliser un `Set` ou `Sorted Set` pour indexer un champ non-clé primaire. Ex : `ZADD idx:age 25 user:42` permet un range scan par âge.

 The picture can't be displayed.

2

Relations 1-N & N-N

Stocker les IDs liés dans un `Set` (non ordonné) ou `Sorted Set` (ordonné par score). La dénormalisation est souvent préférable aux jointures.

 The picture can't be displayed.

3

Pagination

`ZRANGEBYSCORE` ou `ZRANGE ... BYSCORE LIMIT offset count` pour une pagination cursor-based efficace sans `SKIP`.

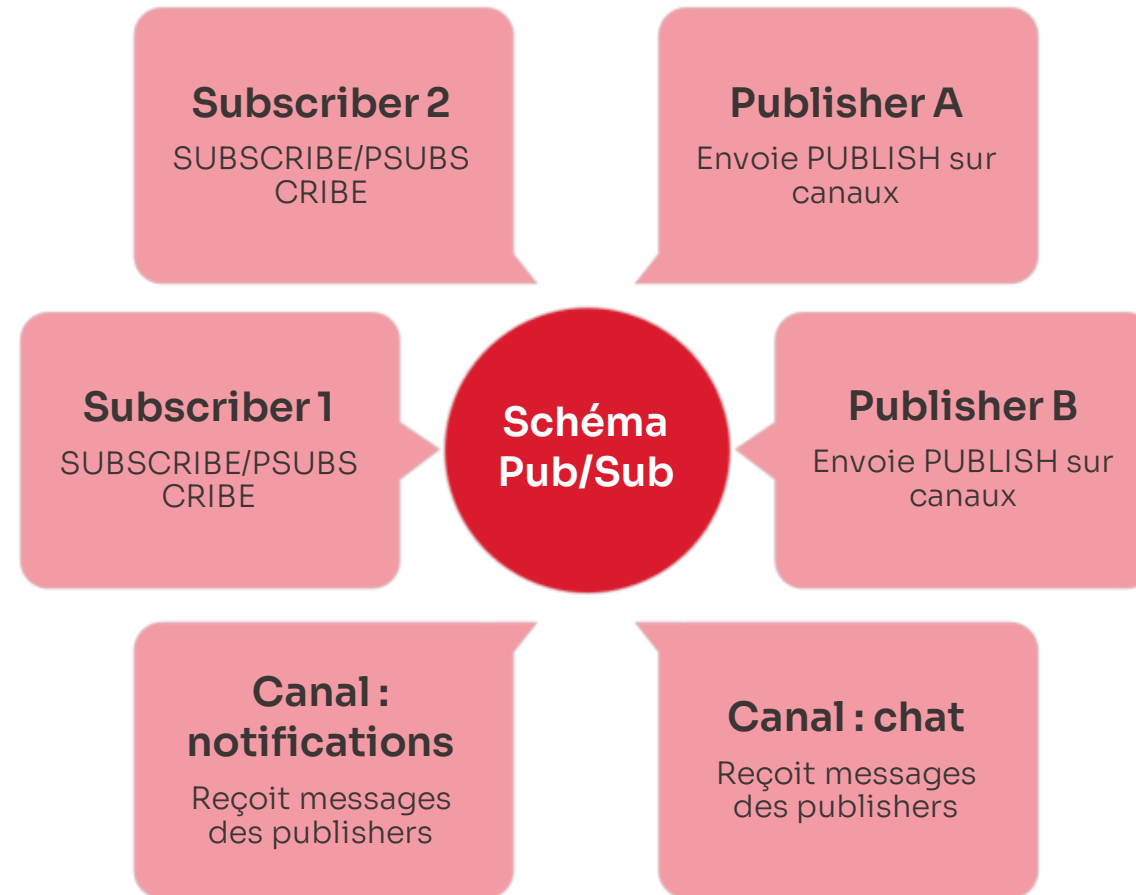
 The picture can't be displayed.

4

Compteurs Multi-dimensionnels

Combiner des Hashes ou `Sorted Sets` pour des métriques croisées : vues par produit × pays × heure, sans base SQL.

Publish / Subscribe : Architecture



Les commandes clés : `PUBLISH canal message`, `SUBSCRIBE canal`, `PSUBSCRIBE motif*`. Le modèle est **fire-and-forget** : aucun message n'est persisté. Un message publié sans abonné actif est définitivement perdu.

Pub/Sub : Cas d'Usage & Limites Critiques

✓ Cas d'usage adaptés

- Invalidation de cache distribué
- Indicateurs de présence (online/offline)
- Chat en temps réel (tolérant aux pertes)
- Synchronisation WebSocket multi-instances
- Keyspace notifications (`__keyevent@0__:expired`)

✗ Limites & dangers

- Aucune persistance — reconnexion = messages perdus
- Client lent → déconnexion par `client-output-buffer-limit`
- Pas d'accusé de réception ni de rejeu
- Interdit pour messages critiques → utiliser Streams
- Keyspace notifs : surcoût CPU, déconseillé en prod haute charge

✗ Ne jamais utiliser Pub/Sub pour des événements métier critiques (paiements, commandes, alertes de sécurité).
Préférer les Streams Redis.

STREAMS

Redis Streams : Journal Append-Only

Inspirés de Apache Kafka, les Streams Redis sont un journal append-only persistant. Chaque entrée possède un ID `timestamp-séquence` unique, garantissant l'ordre total. Contrairement à Pub/Sub, les messages survivent à la déconnexion des consommateurs.

XADD

Ajoute une entrée avec des champs clé-valeur. `XADD stream *
champ valeur`

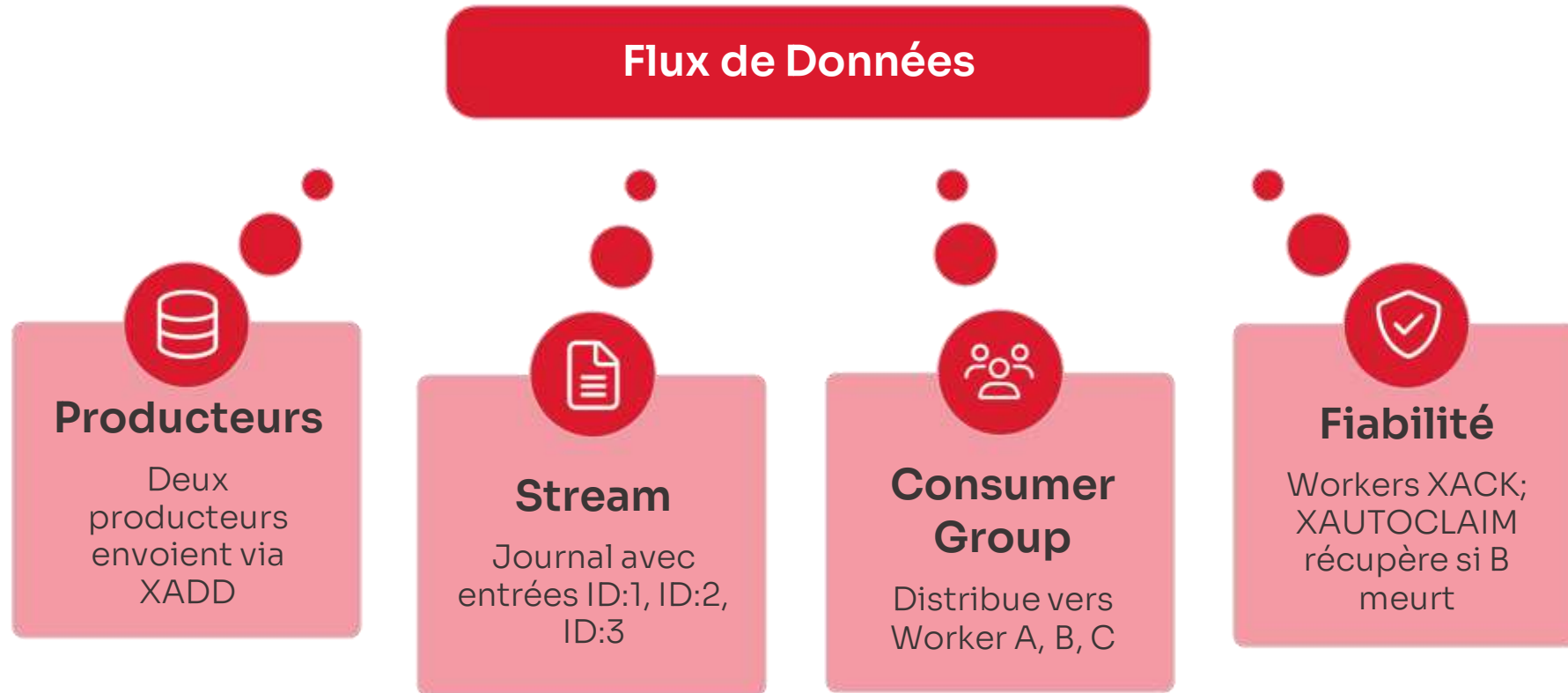
XRANGE

Lit une plage d'entrées par ID.
Lecture historique depuis le début.

XREAD BLOCK

Lecture bloquante — attend de nouveaux messages comme un *tail -f*.

Consumer Groups : Distribution & Fiabilité



Les consumer groups garantissent qu'aucun message n'est perdu si un worker tombe. `XPENDING` liste les messages non acquittés, `XAUTOCLAIM` les réaffecte automatiquement après un timeout.

Commandes Consumer Groups



```
XGROUP CREATE stream groupe $  
MKSTREAM
```

Crée un groupe de consommateurs. \$ = seulement les nouveaux messages ; 0 = depuis le début du stream.



```
XACK stream groupe id
```

Acquitte le traitement — retire le message de la Pending Entry List (PEL).



```
XREADGROUP GROUP g consumer BLOCK 0  
COUNT 10 STREAMS s >
```

Lecture exclusive : > signifie "messages non encore délivrés à ce groupe". Chaque message est verrouillé pour ce consommateur.



```
XAUTOCLAIM stream groupe consumer  
min-idle-time 0
```

Réclame les messages idle depuis plus de N ms — essentiel pour la tolérance aux pannes.

Borner la Croissance d'un Stream

MAXLEN approximatif

La tilde ~ autorise Redis à tronquer par blocs entiers (listpacks), ce qui est bien plus efficace qu'un trimming exact à chaque écriture.

```
XADD events MAXLEN ~ 10000 * type click
```

Conserve environ 10 000 entrées en sacrifiant une précision exacte — compromis performances/rétention optimal.

XTRIM manuel

Permet de déclencher le trimming à la demande, indépendamment des XADD. Utile pour les jobs de maintenance périodiques.

```
XTRIM events MAXLEN ~ 5000
```

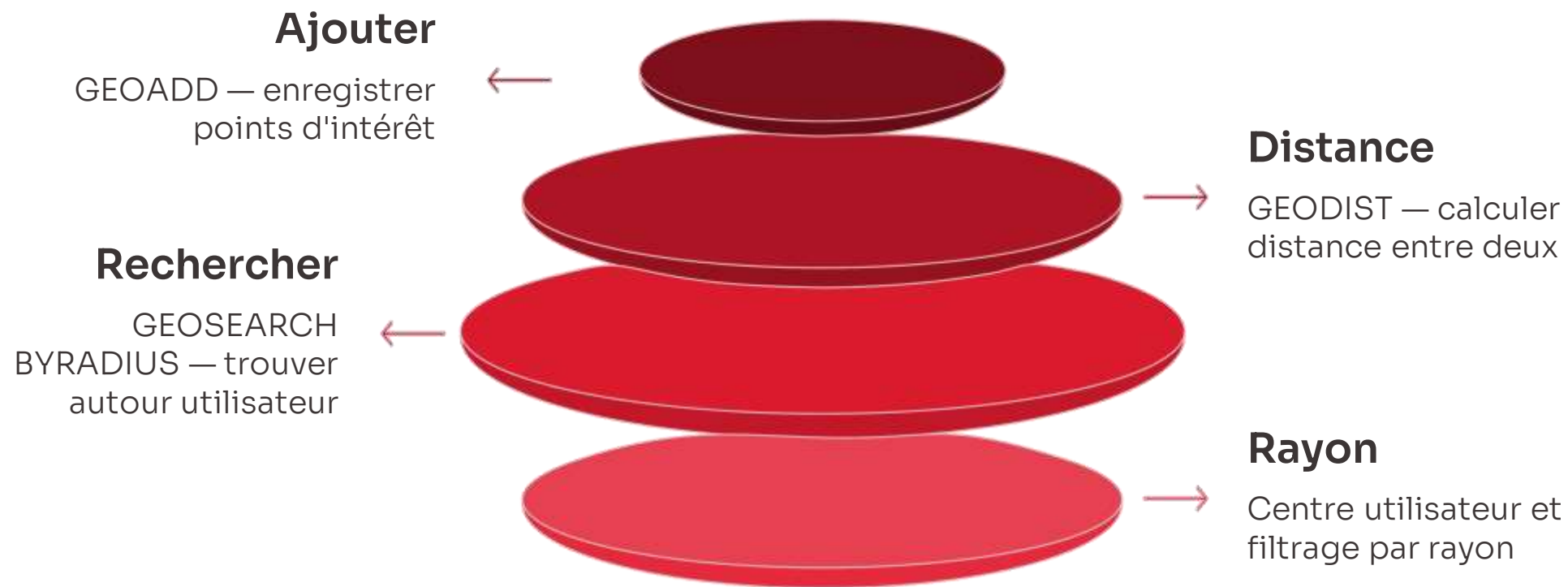
Combiner avec un TTL ou un job cron pour une gestion fine de la rétention selon les besoins métier.

Pub/Sub vs List vs Stream

Critère	Pub/ Sub	List (LPUSH/ BRPOP)	Stream
Persistence	Non	Oui	Oui
Accusé de réception	Non	Non	XACK
Rejeu historique	Non	Non	Oui
Consumer groups	Non	Manuel	Natif
Multi-consommateurs	Fan-out	1 seul	Fan-out + groups
Latence	Très faible	Faible	Faible

📌 Règle pratique : Pub/Sub pour la diffusion temps-réel tolérante aux pertes, List pour les queues simples mono-consommateur, Stream pour tout ce qui nécessite durabilité et traitement garanti.

Géolocalisation avec Redis



Les données géospatiales sont stockées dans un **Sorted Set** encodé en geohash 52 bits. Toutes les commandes **Z*** sont donc applicables. **GEORADIUS** est dépréciée depuis Redis 6.2 — utiliser **GEOSEARCH**.

Commandes Géospatiales

GEOADD

```
GEOADD lieux 2.3488 48.8534 "Paris" 5.3698 43.2965  
"Marseille"
```

Ajoute un ou plusieurs membres avec leur longitude et latitude.

GEODIST

```
GEODIST lieux Paris Marseille km
```

Retourne la distance sphérique entre deux membres.

Unités : m, km, mi, ft.

GEOSEARCH

```
GEOSEARCH lieux FROMLONLAT 2.35 48.85 BYRADIUS 50  
km ASC COUNT 10
```

Recherche dans un rayon ou rectangle. Remplace GEORADIUS.

GEOSEARCHSTORE

Stocke les résultats d'une recherche géospatiale dans une nouvelle clé Sorted Set pour réutilisation ou pagination.

HyperLogLog : Comptage Probabiliste

Principe

Structure probabiliste qui estime la cardinalité d'un ensemble sans stocker les éléments individuels. Mémoire fixe de 12 Ko quel que soit le volume, avec une erreur standard de $\pm 0,81$ %.

- `PFADD visiteurs user:42`
- `PFCOUNT visiteurs`
- `PFMERGE total h111 h112`

Quand utiliser ?

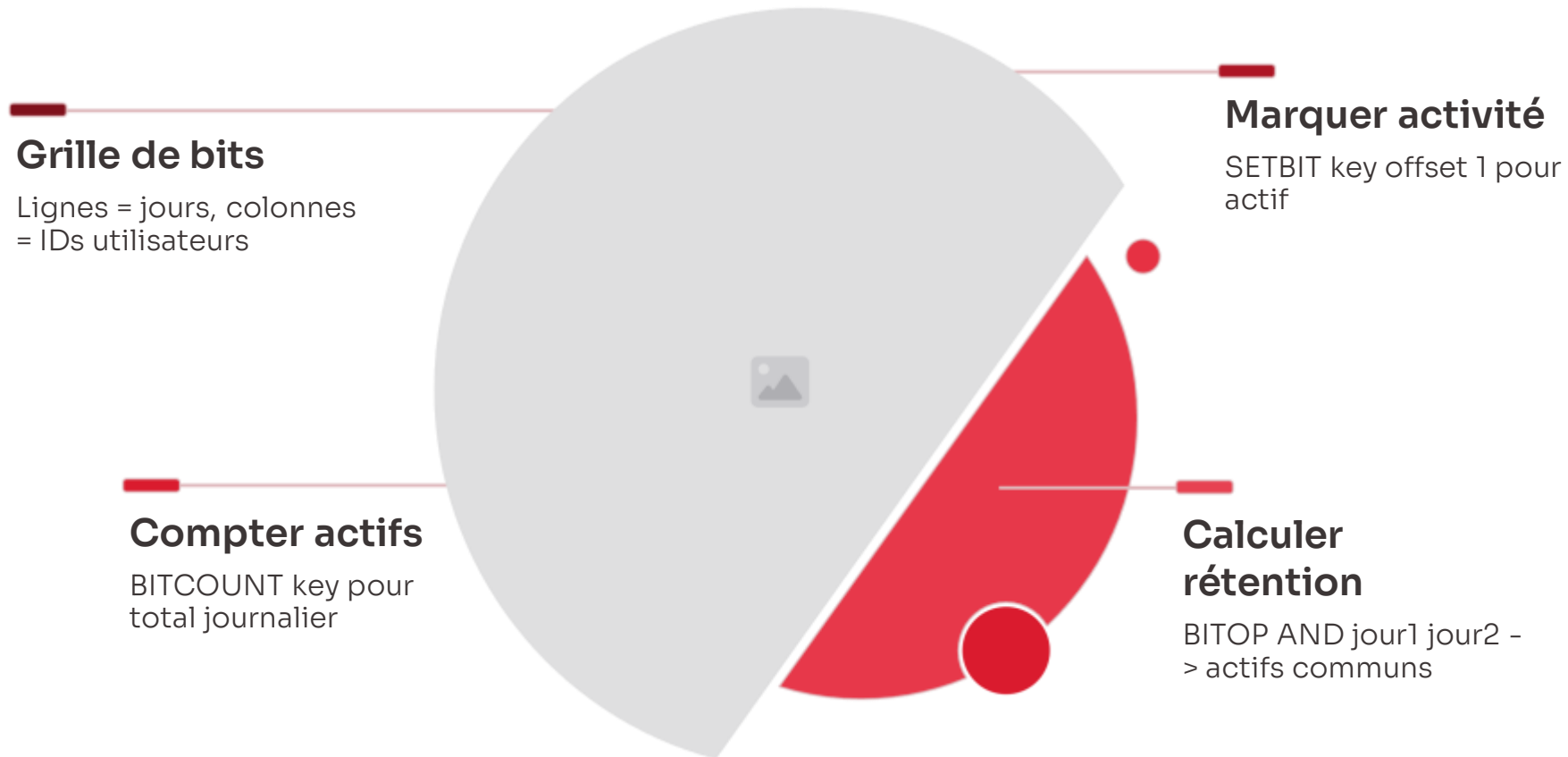
HyperLogLog → ordre de grandeur sur des milliards d'éléments distincts (visiteurs uniques, sessions, événements).

Set classique → appartenance exacte (*cet utilisateur a-t-il vu cet article ?*) ou comptage précis requis.



Cas typique : compter les visiteurs uniques quotidiens d'un site à fort trafic. 12 Ko vs potentiellement plusieurs Go avec un Set.

BitMap : Présence & Rétention à Coût Minimal



1 million d'utilisateurs = seulement 125 Ko par jour. Condition indispensable : les identifiants doivent être des entiers denses (offset = ID utilisateur). Si les IDs sont des UUID ou éparés, le BitMap devient contre-productif.

Commandes BitMap



SETBIT / GETBIT

```
SETBIT actifs:2024-01-15 42 1
```

Marque/lit le bit à l'offset. O(1) garanti.



BITCOUNT

```
BITCOUNT actifs:2024-01-15
```

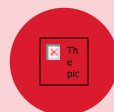
Compte les bits à 1 (optionnellement sur une plage d'octets).



BITOP

```
BITOP AND retention lundi mardi
```

AND/OR/XOR/NOT entre BitMaps. Calcul de rétention en une commande.



BITPOS

```
BITPOS actifs:2024-01-15 1
```

Trouve la position du premier bit à 0 ou 1 — utile pour l'attribution d'IDs.

Requêtes Complexes & Index Manuels

Redis ne dispose pas de moteur de requêtes ad hoc. Toute recherche complexe doit être anticipée lors de la modélisation.

Index manuels avec Sets/Sorted Sets

Maintenir des index secondaires manuellement à chaque écriture. Requiert une discipline stricte de mise à jour transactionnelle (`MULTI/EXEC` ou Lua) pour éviter les désynchronisations.

Scripts Lua

Exécuter des traitements complexes côté serveur de manière atomique. Idéal pour les logiques conditionnelles multi-clés. Limiter la durée : un script long bloque le thread principal.

RediSearch (Redis Stack)

Module dédié aux recherches full-text, filtres multi-attributs, agrégations et requêtes géospatiales avancées. Indispensable dès que la complexité dépasse ce que les Sorted Sets permettent.

Anti-Patterns à Bannir en Production

The picture can't be displayed.

⊘ KEYS * en production

KEYS pattern bloque le thread principal pendant le scan complet du keyspace — catastrophique sur des millions de clés. Remplacer systématiquement par `SCAN cursor MATCH pattern COUNT 100` qui est incrémental et non-bloquant.

The picture can't be displayed.

⊘ Big Keys

Une clé contenant un Hash de 10 millions de champs ou une List de 5 millions d'éléments crée des latences lors des opérations de lecture/écriture et bloque la réplication. Diagnostiquer avec `redis-cli --bigkeys` ou `OBJECT FREQ`.

The picture can't be displayed.

⊘ Hot Keys

Une clé unique concentrant 80 % des requêtes saturera un seul nœud en cluster. Solutions : réplication locale (`READONLY` sur réplicas), sharding applicatif avec suffixe aléatoire, ou cache local L1 côté application.

TP : Spécification des Composants



Visiteurs Uniques — HyperLogLog

`PFADD visiteurs:{date} user:{id}` à chaque visite.
`PFCOUNT` pour le total. `PFMERGE` pour la semaine. Erreur <1%, mémoire fixe.



Présence Active — BitMap

`SETBIT presence:{date} {userId} 1`. `BITCOUNT` pour les actifs du jour. `BITOP AND` pour la rétention `JONJ1`.



Top Produits — Sorted Set

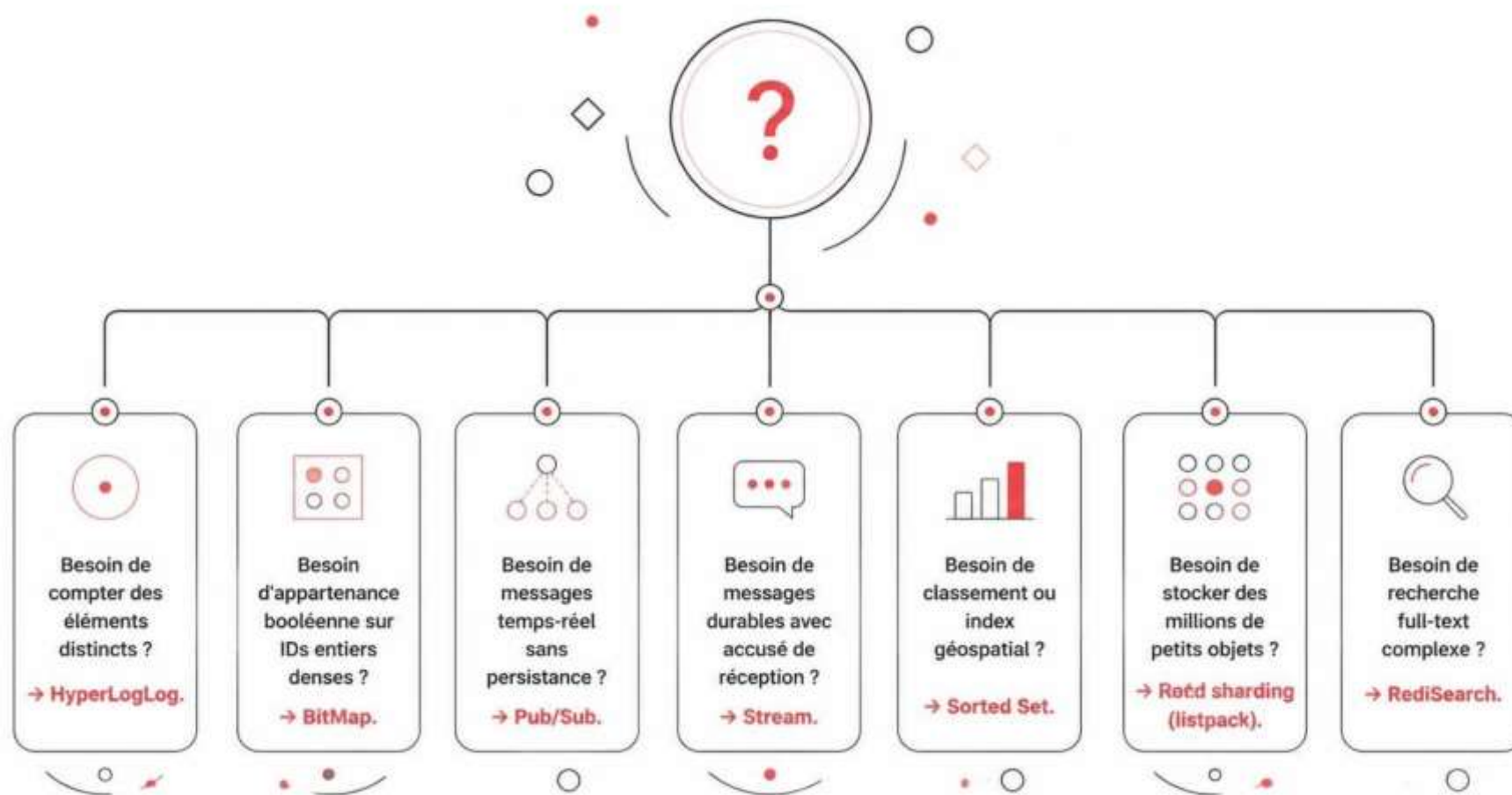
`ZINCRBY top:produits 1 prod:{id}` à chaque vue.
`ZRANGE ... REV LIMIT 0 10` pour le top 10 instantané sans tri applicatif.



File Événements Fiable — Stream

`XADD events * type achat produit {id}`. Consumer group de workers pour traitement garanti avec `XACK`.
`MAXLEN ~ 100000` pour borner la rétention.

Synthèse : Choisir la Bonne Structure



Points Clés à Retenir

Encodages internes

Surveiller les seuils de conversion avec `OBJECT ENCODING` — le hash sharding peut réduire la RAM de 80 %.

Pipeline & multi-arg

Réduire les round-trips réseau est souvent le gain de performance le plus rapide à obtenir.

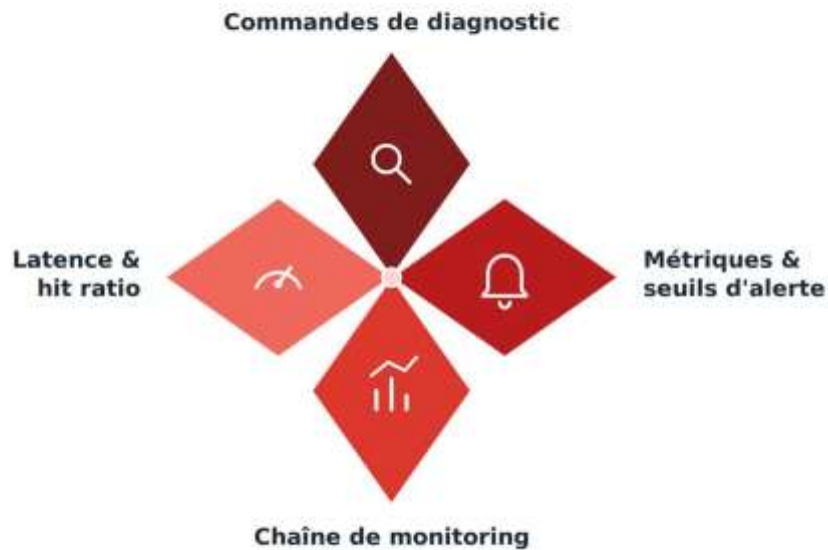
Pub/Sub $\neq$ Streams

Fire-and-forget pour la diffusion temps-réel ; Streams pour la durabilité et la garantie de traitement.

Structures spécialisées

HLL pour la cardinalité, BitMap pour la présence, Geo pour la proximité — choisir l'outil selon la sémantique, pas l'habitude.

📌 Prochain module : Redis Cluster, haute disponibilité et Sentinel — architecture de production à l'échelle.



Module 7 : Monitoring du trafic Redis

Maîtriser l'observabilité de Redis : latence, mémoire, connexions et hit ratio. Ce module couvre les commandes natives de diagnostic, les métriques clés avec leurs seuils d'alerte, et l'intégration dans une chaîne de monitoring moderne.

MODULE 7

MONITORING

PERFORMANCE

Pourquoi monitorer Redis ?

Redis est un composant critique dans la plupart des architectures modernes. Sans observabilité, un incident peut passer inaperçu jusqu'à dégradation sévère. Quatre axes fondamentaux doivent être surveillés en permanence.



Latence

Détecter les commandes lentes qui bloquent le thread principal et dégradent l'expérience utilisateur.



Mémoire

Surveiller la consommation par rapport à `maxmemory` pour anticiper les évictions et OOM.



Connexions

Contrôler le nombre de clients connectés, bloqués, et les connexions rejetées.



Hit Ratio

Mesurer l'efficacité du cache : un ratio faible indique un TTL inadapté ou un cache sous-dimensionné.

MONITOR : flux temps réel

Utilisation et précautions

La commande `MONITOR` diffuse en temps réel toutes les commandes reçues par Redis. C'est un outil de débogage puissant, mais à utiliser avec une grande prudence.

❌ Jamais en production prolongée. `MONITOR` peut doubler la charge CPU et saturer le réseau. Réservez-le aux sessions de débogage courtes et ciblées.

- Affiche chaque commande avec timestamp, client source et arguments
- Utile pour capturer une session applicative suspecte
- À arrêter immédiatement après diagnostic (`CTRL+C`)

```
$ redis-cli MONITOR
OK
1704067200.123456 [0 127.0.0.1:52341] "GET"
"user:42"
1704067200.124001 [0 127.0.0.1:52341] "SET"
"session:abc" "data" "EX" "3600"
1704067200.125312 [0 127.0.0.1:52342]
"HGETALL" "product:catalog"
1704067200.126100 [0 127.0.0.1:52341]
"KEYS" "*"
1704067200.127890 [0 127.0.0.1:52343]
"LPUSH" "queue:tasks" "job1"
```

INFO : radiographie complète du serveur

La commande `INFO` retourne un rapport structuré en sections. Chaque section cible un domaine précis de l'état interne de Redis.

```
$ redis-cli INFO memory
# Memory
used_memory:1048576
used_memory_human:1.00M
used_memory_rss:2097152
maxmemory:8589934592
maxmemory_human:8.00G
mem_fragmentation_ratio:1.23
mem_allocator:libc

$ redis-cli INFO replication
# Replication
role:master
connected_slaves:2
master_replid:a1b2c3d4e5
master_repl_offset:123456
```

Sections disponibles

server

Version, OS, uptime, config

clients

Connexions actives,
bloquées

memory

Usage RAM, fragmentation

persistence

RDB/AOF statuts, dernière
sauvegarde

stats

Ops/sec, hits, misses,
évictions

keyspace

Clés par DB, TTL, expirations

SLOWLOG : premier réflexe face aux latences

Le **SLOWLOG** enregistre toutes les commandes dépassant un seuil configurable. C'est l'outil numéro un pour identifier les commandes pathologiques sans impacter les performances.

Configuration et lecture

```
# redis.conf
slowlog-log-slower-than 10000
# (microsecondes, 10ms par défaut)
slowlog-max-len 128

# Lecture des entrées
$ redis-cli SLOWLOG GET 5
1) 1) (integer) 42          # ID
    2) (integer) 1704067200 # timestamp
    3) (integer) 35421      # durée µs
    4) 1) "KEYS"
        2) "*"
2) 1) (integer) 41
    2) (integer) 1704067180
    3) (integer) 28700
    4) 1) "HGETALL"
        2) "product:catalog"
```

Coupables fréquents

 The picture can't be displayed.

KEYS *

Scan bloquant de toutes les clés — remplacer par **SCAN** avec curseur.

 The picture can't be displayed.

HGETALL géant

Hash à des milliers de champs — paginer avec **HSCAN**.

 The picture can't be displayed.

Scripts Lua longs

Un script Lua bloque le serveur entier pendant son exécution.

Diagnostic de latence avancé

Commandes de diagnostic

```
# Latence en temps réel (1 mesure/sec)
$ redis-cli --latency -h 127.0.0.1 -p 6379
min: 0, max: 1, avg: 0.10 (1000 samples)

# Latence intrinsèque du système (OS/hardware)
$ redis-cli --intrinsic-latency 30
Max latency so far: 1 microseconds
Max latency so far: 9 microseconds
Max latency so far: 47 microseconds

# Historique des pics de latence
$ redis-cli LATENCY HISTORY event
$ redis-cli LATENCY LATEST

# Diagnostic automatique
$ redis-cli LATENCY DOCTOR
```

Interprétation des outils

1 --latency

Mesure le RTT réseau + traitement Redis. Idéal pour détecter une dégradation progressive.

2 --intrinsic-latency

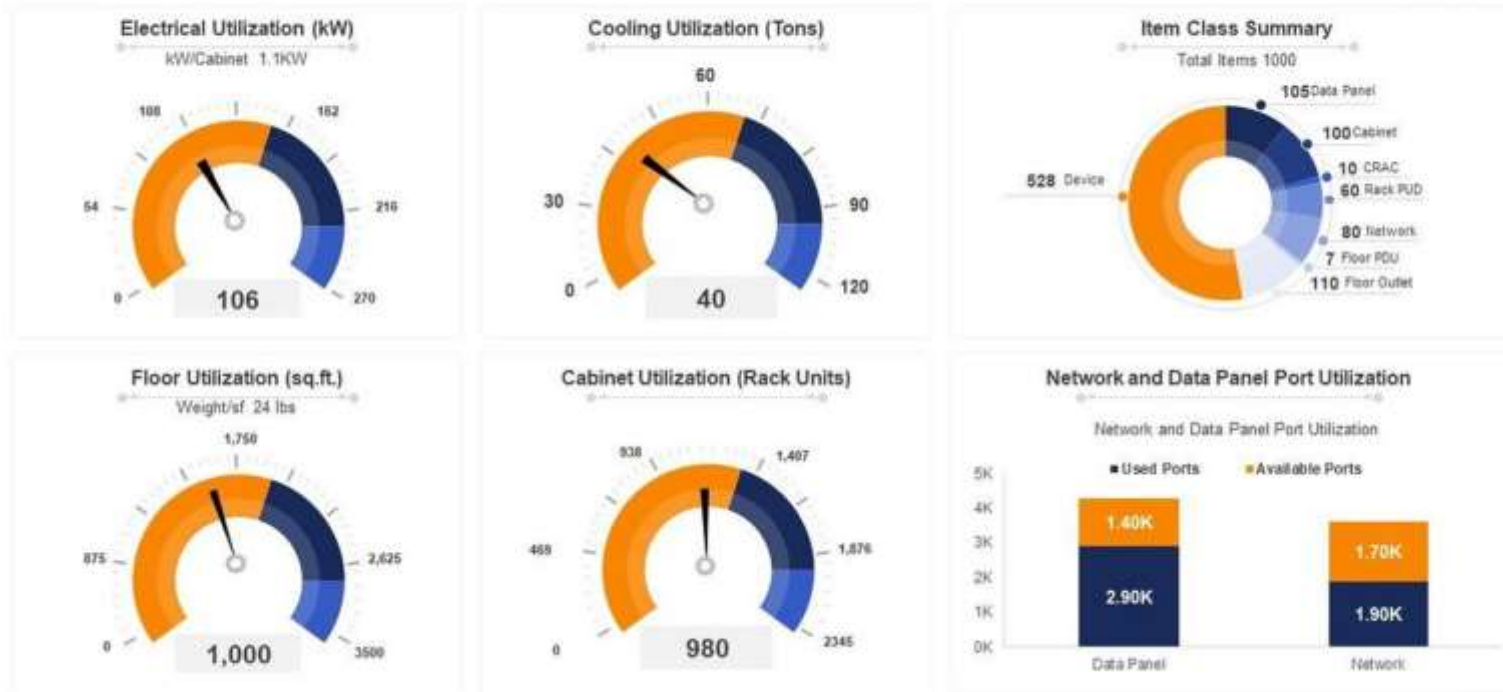
Isole la latence du système d'exploitation (scheduler, swapping). À comparer avec `--latency`.

3 LATENCY DOCTOR

Analyse automatique avec recommandations textuelles. Point d'entrée idéal pour un débutant.

Data Center Dashboard to Monitor and Optimize Performance

March 2022



This graph/chart is linked to excel and changes automatically based on data. Just left click on it and select "Edit Data".

Métriques clés et seuils d'alerte

Un ensemble de métriques essentielles à surveiller en production, avec des seuils concrets pour déclencher vos alertes.

Tableau des métriques critiques

Métrique	Source INFO	Seuil d'alerte	Interprétation
used_memory / maxmemory	memory	> 80%	Risque d'éviction imminente
evicted_keys	stats	Croissance continue	Politique d'éviction active, TTL à revoir
hit_ratio (hits/hits+misses)	stats	< 80%	TTL trop court ou cache sous-dimensionné
connected_clients	clients	Pic anormal	Vérifier pool de connexions applicatif
blocked_clients	clients	> 0 persistant	BLPOP/BRPOP en attente prolongée
rejected_connections	stats	> 0	maxclients atteint, urgence
mem_fragmentation_ratio	memory	> 1,5	Fragmentation mémoire élevée, ACTIVEDEFRAG
rdb_last_bgsave_status	persistence	≠ ok	Sauvegarde RDB échouée, perte de données possible
master_link_status	replication	≠ up	Réplica déconnecté du master

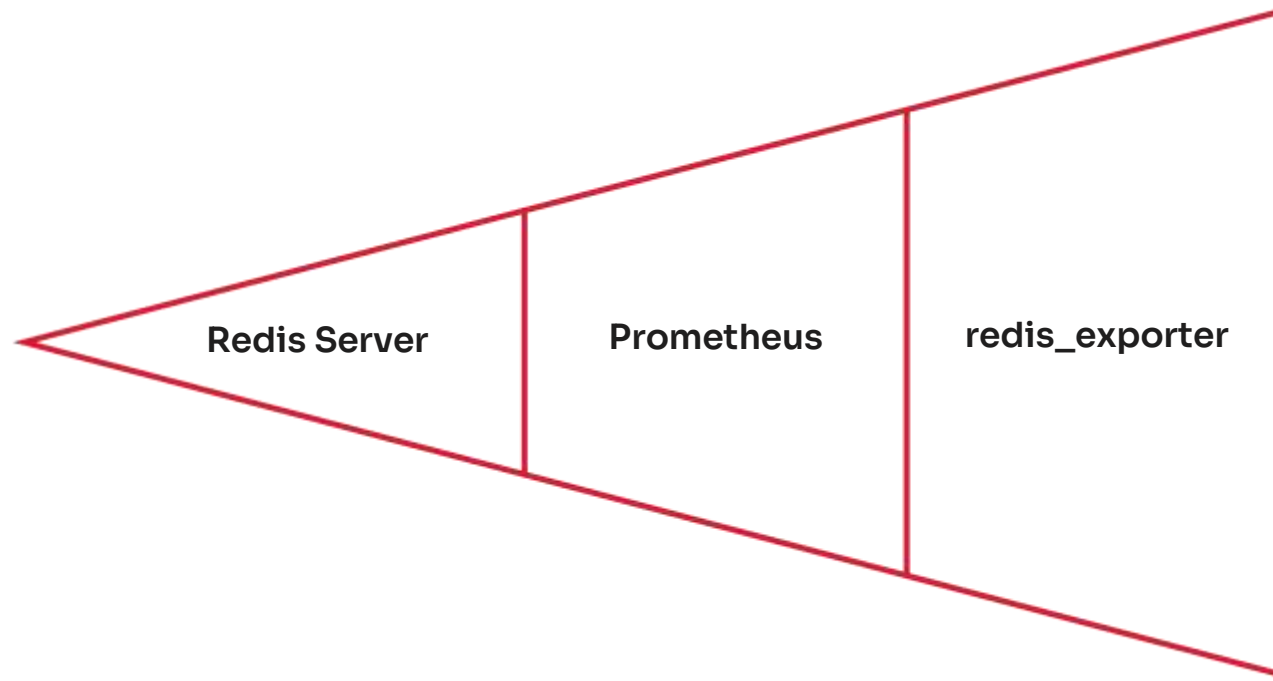
Outils du marché : comparatif

Plusieurs solutions couvrent le monitoring Redis, des outils natifs aux plateformes SaaS. Choisissez selon votre infrastructure existante et vos contraintes d'exploitation.

Outil	Type	Pointsforts	Limites	Coût
RedisInsight	GUI native	Profileur intégré, analyse mémoire, browser de clés	Single-instance, pas d'historique long terme	Gratuit
Prometheus+ redis_exporter	Open-source	Métriques granulaires, alerting flexible, self-hosted	Configuration manuelle, stockage à gérer	Gratuit
Grafana	Visualisation	Dashboards riches, multi-sources, alerting natif	Nécessite une source de données (Prometheus)	Gratuit / Cloud
Datadog	SaaS	Intégration Redis clé en main, APM, corrélation logs	Coût élevé à grande échelle	Payant
New Relic	SaaS	Observabilité full-stack, anomaly detection IA	Courbe d'apprentissage, pricing complexe	Payant
CloudWatch	AWS natif	Intégration ElastiCache native, zéro infrastructure	Limité à l'écosystème AWS	Pay-as-you-go

Chaîne de monitoring : Redis → Prometheus → Grafana

Architecture de référence pour un monitoring production robuste et open-source. Chaque composant joue un rôle précis dans la collecte, le stockage et la visualisation des métriques.



```
# prometheus.yml - configuration scrape
scrape_configs:
  - job_name: 'redis'
    static_configs:
      - targets: ['localhost:9121']
    scrape_interval: 15s
```

```
# Lancement redis_exporter
$ docker run -d \
  -p 9121:9121 \
  oliver006/redis_exporter \
  --redis.addr=redis://localhost:6379
```

Synthèse

Points clés à retenir



MONITOR

Débogage uniquement, jamais en production prolongée. Impact CPU critique.



Hit ratio

En dessous de 80% : revoir TTL ou dimensionner le cache.



SLOWLOG

Premier réflexe face à toute latence inexpliquée. Traque `KEYS`, `HGETALL` et Lua.

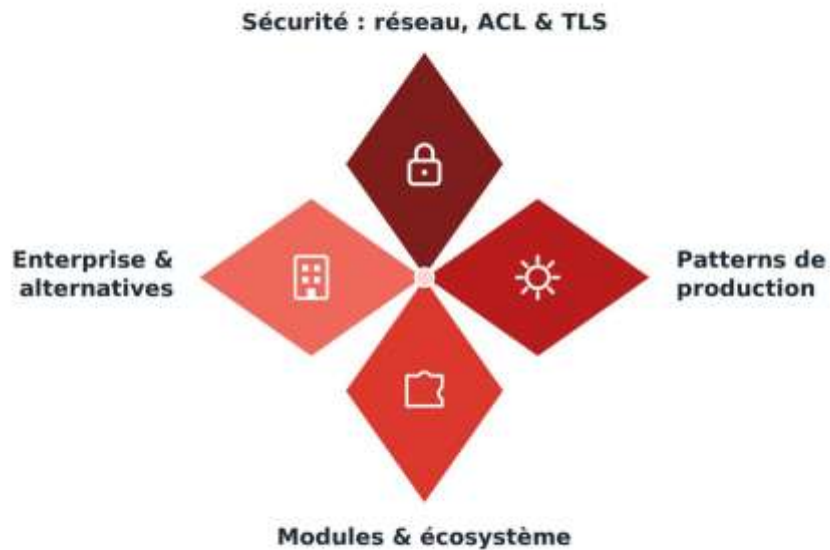


Chaîne recommandée

redis_exporter → Prometheus → Grafana pour un monitoring robuste et pérenne.



Un hit ratio inférieur à 80% est le signal le plus fréquemment négligé en production. Surveillez-le en continu.



Module 8 : Sécurité, Patterns de Production & Écosystème Redis

De la sécurisation réseau à l'écosystème avancé — tout ce qu'il faut savoir pour déployer Redis en production avec confiance, robustesse et performance.

MODULE 8 · FORMATION REDIS

Postulat fondamental

Redis n'est pas conçu pour Internet ouvert

Le risque en chiffres

Un Redis exposé sur Internet sans authentification est compromis en quelques minutes. Des scanners automatiques scrutent en permanence les ports 6379 ouverts à la recherche de serveurs vulnérables.

Redis a été conçu pour fonctionner dans un réseau de confiance, derrière un pare-feu, accessible uniquement aux applications autorisées.

Vecteurs d'attaque courants

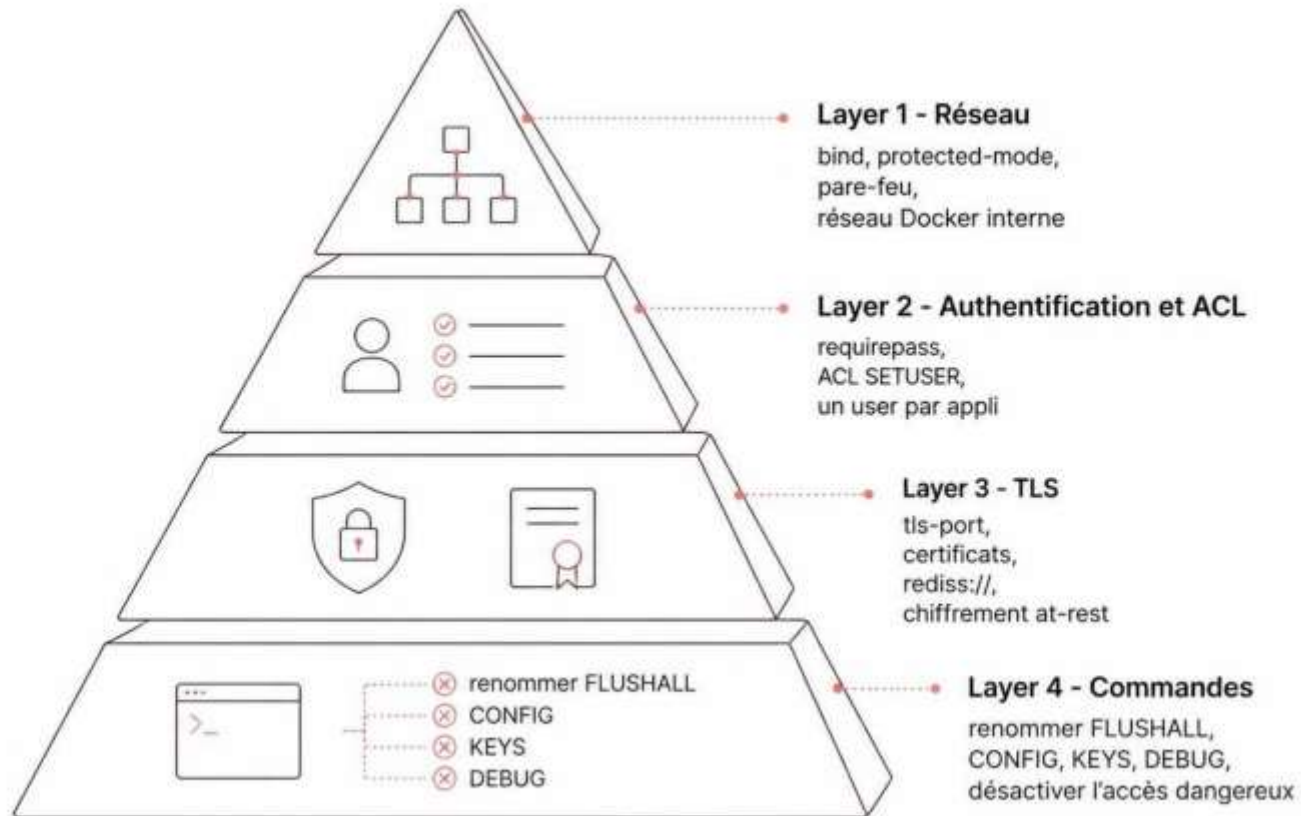
- Lecture de données sensibles en clair
- Injection de clés malveillantes (webshells)
- Utilisation comme proxy ou relais d'attaque
- Exécution de commandes système via `CONFIG SET dir`
- Effacement total de données via `FLUSHALL`



Un Redis non sécurisé accessible depuis Internet est une faille critique, pas un risque théorique.

Les 4 couches de sécurisation Redis

La défense en profondeur consiste à empiler plusieurs couches indépendantes : la compromission d'une couche ne suffit pas à accéder aux données.



Durcissement Réseau

`bind`

Ne jamais utiliser `0.0.0.0`. Lier Redis uniquement à l'interface locale ou interne : `bind 127.0.0.1` ou l'IP privée du service.

`protected-mode`

Activé par défaut. Refuse les connexions externes si aucun mot de passe ni `bind` explicite n'est configuré. Ne jamais désactiver.

Pare-feu

Bloquer le port 6379 au niveau OS (iptables, nftables, Security Groups AWS/GCP) pour n'autoriser que les IPs/CIDRs des applications clientes.

Réseau Docker

Placer Redis sur un réseau Docker interne (`--network backend`). Ne jamais publier le port vers l'hôte (`-p 6379:6379`) en production.

Authentication & ACL Redis 6+

Authentication classique

`requirepass <motdepasse>` — protection minimale, un seul utilisateur global. Suffisant pour les environnements simples, mais limité.

ACL : un utilisateur par application

Redis 6 introduit les ACL permettant de définir des utilisateurs avec des droits granulaires :

- `~motif:*` — restriction par préfixe de clé
- `+commande` — autoriser une commande
- `-@dangerous` — exclure une catégorie
- `+@read` — lecture seule pour la supervision

Commandes dangereuses à renommer ou désactiver

 The picture can't be displayed.

FLUSHALL / **FLUSHDB**

Supprime toutes les données. Renommer en chaîne aléatoire ou désactiver via `rename-command FLUSHALL ""`.

 The picture can't be displayed.

CONFIG

Permet de modifier la config à chaud, d'écrire des fichiers. Accès réservé aux admins uniquement.

 The picture can't be displayed.

KEYS / **DEBUG**

KEYS * bloque Redis entier. **DEBUG** expose des vecteurs d'exploitation. À désactiver en prod.

TLS & Chiffrement

Transport chiffré (TLS)

Depuis Redis 6, le support TLS est natif. Configuration côté serveur :

- `tls-port 6380` — port dédié TLS
- `tls-cert-file`, `tls-key-file`, `tls-ca-cert-file`
- Côté client : schéma `rediss://` (double s)
- Vérification du certificat serveur obligatoire en prod

Chiffrement at-rest

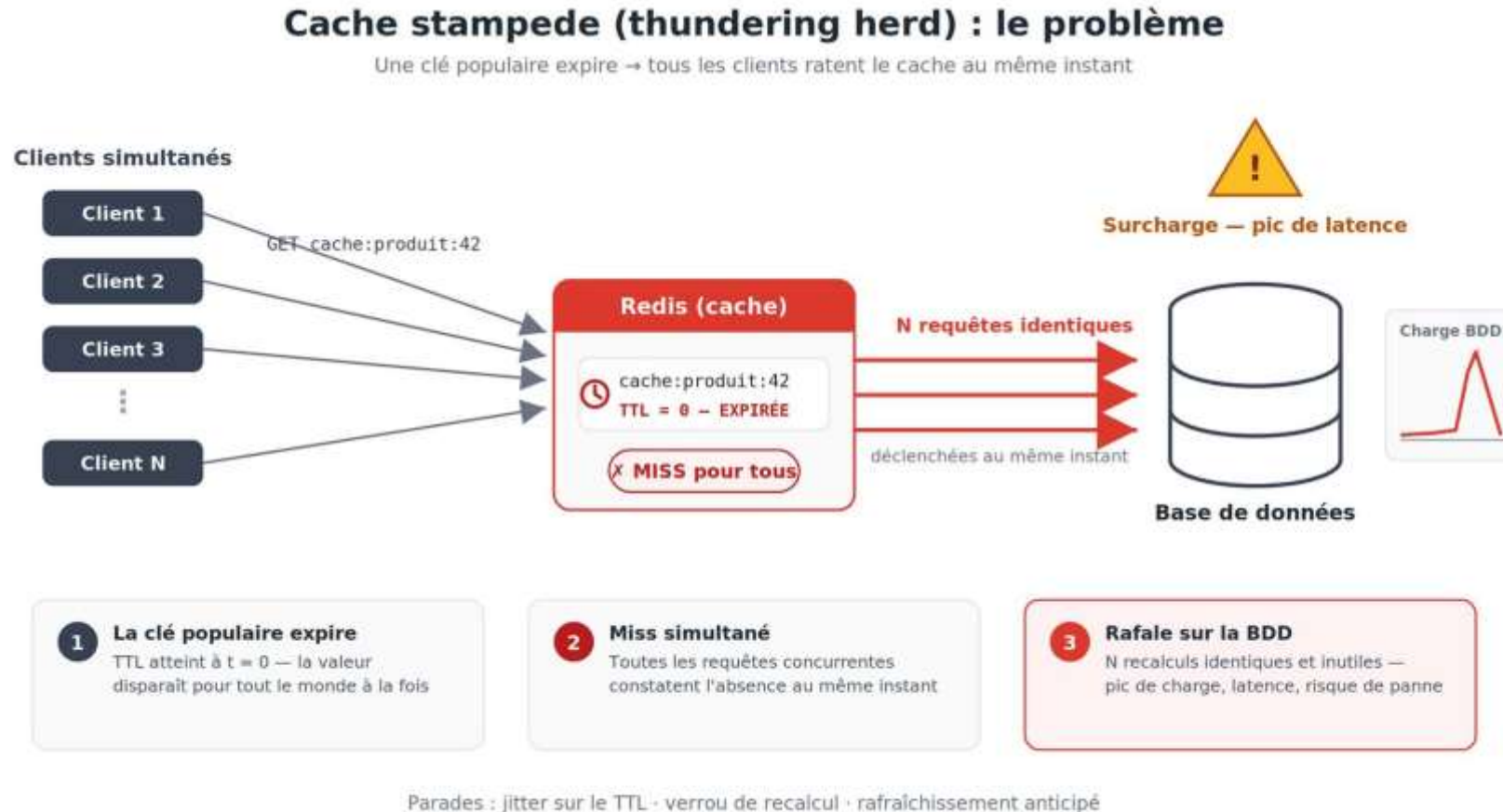
Redis OSS ne chiffre pas les données sur disque. Le chiffrement at-rest est disponible via Redis Enterprise ou les solutions cloud managées (ElastiCache, Memorystore).

Checklist TLS en production

- ✓ Utiliser des certificats signés (Let's Encrypt ou PKI interne)
- ✓ Activer la vérification mutuelle (mTLS) pour les services critiques
- ✓ Désactiver TLS 1.0/1.1 — exiger TLS 1.2 minimum
- ☐ Rotation régulière des certificats automatisée
- ☐ Surveiller les dates d'expiration des certificats

Cache Stampede : Le Problème

Le **cache stampede** (ou **thundering herd**) se produit quand une clé populaire expire simultanément pour de nombreux clients, déclenchant une rafale de requêtes simultanées vers la base de données.



Cache Stampede : Les 3 Parades

The picture can't be displayed.

Jitter sur le TTL

Ajouter une valeur aléatoire au TTL pour disperser les expirations dans le temps :

```
TTL = base + random(0, jitter)
```

Simple, sans overhead. Évite les expirations synchronisées sur les caches populaires.

The picture can't be displayed.

Verrou de recalcul

Seul le premier thread obtient un verrou (SET NX PX) pour recalculer la valeur. Les autres attendent ou retournent une valeur périmée (*stale*).

Élimine la rafale, mais introduit une latence pour les clients en attente.

The picture can't be displayed.

Rafraîchissement anticipé

Rafraîchir la clé *avant* son expiration, de manière asynchrone, lorsque le TTL descend sous un seuil. La valeur reste toujours en cache.

Meilleure expérience utilisateur, mais nécessite un processus de refresh en arrière-plan.

Verrou Distribué Redis : Implémentation & Limites

Implémentation correcte

- `SET key jeton NX PX ttl` — acquisition atomique avec jeton unique (UUID)
- Traitement **plus court** que le TTL du verrou
- Libération via script **Lua atomique** vérifiant le jeton avant DELETE

Le jeton unique garantit qu'un client ne libère pas le verrou d'un autre en cas de lenteur.

Algorithme Redlock

Redlock acquiert le verrou sur la majorité ($N/2+1$) de N Redis indépendants pour tolérer les pannes de nœuds.

Limite honnête : le débat Kleppmann vs Sanfilippo

 The picture can't be displayed.

Fenêtre de double détention

Après un failover en mode répliqué, deux clients peuvent détenir le verrou simultanément (fenêtre de réplication manquante).

 The picture can't be displayed.

Pour les cas critiques

Finance, transactions bancaires : préférer **etcd** ou **ZooKeeper** — systèmes à consensus fort (Raft/Paxos) sans fenêtre de double détention.

Écosystème Applicatif : Ne Pas Réinventer la Roue



BullMQ

File de travaux robuste basée sur Redis. Gère nativement le **retry avec backoff exponentiel**, les **priorités**, les **jobs répétables (cron)**, les **workers concurrents** et les **flux (flows)**. Standard de facto pour les jobs asynchrones Node.js.



connect-redis

Store de sessions Express/Koa utilisant Redis comme backend. Persistance des sessions utilisateur sans état côté serveur — indispensable pour les architectures horizontalement scalables. Configuration en quelques lignes.

- ❏ Avant d'implémenter une file de messages ou un gestionnaire de sessions maison, vérifier l'écosystème existant : les bibliothèques communautaires intègrent des années de cas limites résolus.

Checklist de Mise en Production

✓ À faire absolument

- Configurer `maxmemory` + politique d'éviction cohérente (`allkeys-lru`, `volatile-lru`)
- Appliquer un TTL sur toutes les clés, avec jitter
- Utiliser `SCAN` (jamais `KEYS *`) et `UNLINK (async)`
- Connexion singleton/pool — ne pas créer de client à chaque requête
- Encapsuler les appels Redis dans try/catch avec dégradation gracieuse
- ✓ Superviser : hit ratio, évictions, `SLOWLOG`, mémoire utilisée

⊘ À éviter absolument

`KEYS *` / `FLUSHALL` / `MONITOR` en prod

Bloquent l'event loop. `MONITOR` génère un overhead CPU massif.

Valeurs de plusieurs Mo

Fragmentent la mémoire, ralentissent la sérialisation, dégradent les performances réseau.

Structures non bornées & Pub/Sub critique

Listes/Sets sans `LTRIM`. Pub/Sub ne garantit pas la délivrance — utiliser `BullMQ` pour les messages critiques.

Étendre Redis : L'Architecture des Modules

Redis peut être étendu via des modules natifs qui ajoutent de nouveaux types de données et commandes, chargés dynamiquement dans le processus Redis.



RediSearch

Indexation full-text secondaire, requêtes complexes, recherche vectorielle (KNN) pour l'IA/ML. Alternative à Elasticsearch pour les cas d'usage simples.



RedisTimeSeries

Stockage optimisé de séries temporelles avec agrégations, downsampling automatique et retention configurable. Idéal pour les métriques IoT ou monitoring.



RedisJSON

Stockage et manipulation de documents JSON natifs. Accès partiel aux champs sans désérialiser l'intégralité du document. Compatible JSONPath.



RedisBloom

Structures probabilistes : Bloom Filter, Cuckoo Filter, Count-Min Sketch, HyperLogLog. Déduplication et comptage approximatif à très faible empreinte mémoire.

Ce que Redis ne fait pas

Connaître les limites intrinsèques de Redis est aussi important que ses forces. Redis excelle dans son domaine — ne pas le pousser hors de ses limites.

✗ SQL ad hoc

Pas de langage de requête déclaratif généraliste. RediSearch offre des requêtes structurées mais limitées. Pour du SQL, utiliser PostgreSQL ou MySQL.

✗ Jointures

Aucun support natif des jointures entre entités. Les relations doivent être dénormalisées ou gérées côté application.

✗ ACID multi-clés distribué

Les transactions Redis (MULTI/EXEC) sont atomiques sur un seul nœud mais sans isolation forte ni rollback. Pas d'ACID distribué cross-shards.

✗ Données froides sur disque

Redis est in-memory first. La persistance RDB/AOF n'est pas un stockage disque traditionnel. Pour les données froides volumineuses, utiliser une base relationnelle ou objet.

Redis OSS vs Redis Enterprise

Redis Enterprise ajoute des fonctionnalités critiques pour les déploiements d'entreprise à grande échelle, avec un SLA garanti.

Fonctionnalité	Redis OSS	Redis Enterprise
Réplication	Master/Replica manuelle	Active-Active (CRDT) multi-région
Persistance haute perf.	RDB + AOF sur RAM	Redis on Flash (SSD + RAM)
Multi-tenancy	Séparation par instance	Bases de données multiples isolées
Modules	Installation manuelle	Intégrés et supportés
SLA	Communautaire	99.999% garanti
Support	Forum / GitHub	Support 24/7 dédié

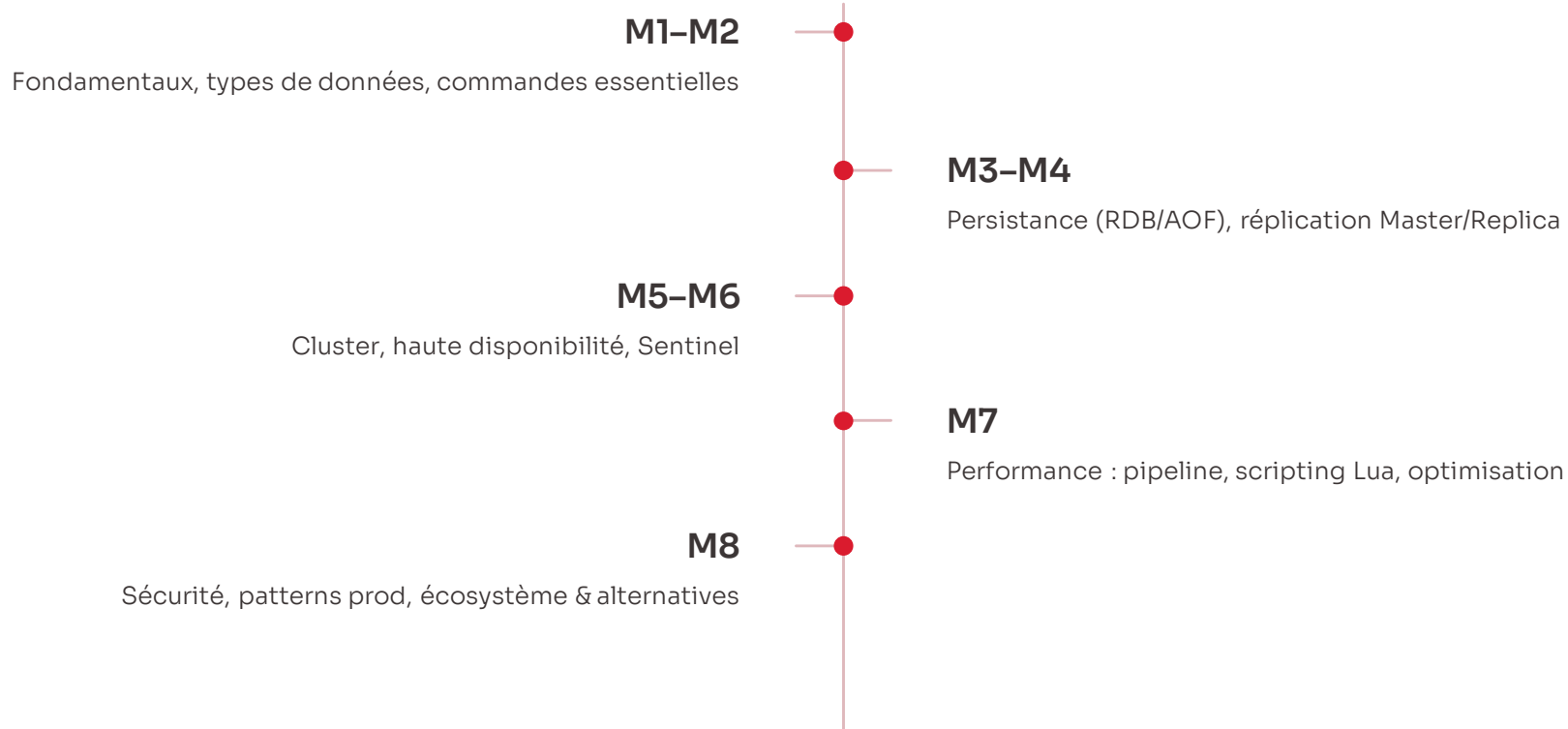
Alternatives à Redis

L'écosystème des caches in-memory s'est enrichi. Selon le contexte (licence, performance, compatibilité), plusieurs alternatives méritent d'être évaluées.

Projet	Base	Licence	Point fort	Cas d'usage
Valkey	Fork Redis 7.2	BSD (Linux Foundation)	Compatible Redis, gouvernance neutre	Migration sans friction depuis Redis OSS
KeyDB	Fork Redis	BSD	Multi-thread natif	Forte concurrence CPU-bound
Dragonfly	Réécriture C++	BSL	25× plus rapide, multi-thread	Très haute performance, faible latence
Memcached	Indépendant	BSD	Simple, ultra-stable, multi-thread	Cache clé/valeur simple sans persistance

📌 Valkey est le successeur naturel de Redis OSS depuis le changement de licence Redis 7.4 vers SSPL. La majorité des clouds majeurs migrent vers Valkey.

Synthèse de la Formation Redis — 8 Modules



Ressources pour continuer

- redis.io — documentation officielle complète
- [Redis University](https://redis.university) — cours gratuits certifiants
- [GitHub redis/redis](https://github.com/redis/redis) — code source et release notes
- valkey.io — fork Linux Foundation

Test de positionnement aval

Le test de positionnement permet de valider les acquis de la formation et d'identifier les axes d'approfondissement individuel. Durée estimée : **20 minutes**.



Merci de votre attention

Place à vos questions et à nos échanges.

FORMATION REDIS · NOSQL

Ressources & références

Documentation officielle

-  **redis.io/docs**
Référence des commandes, guides par langage, exemples
-  **redis.conf commenté**
Référence complète des paramètres de configuration

Clients & frameworks

-  **github.com/redis/jedis**
Wiki Jedis, javadoc, patterns recommandés
-  **github.com/redis/node-redis**
Documentation node-redis v4+, exemples, changelog
-  **spring.io/projects/spring-data-redis**
Documentation Spring Data Redis, auto-configuration

Outils de monitoring

-  **RedisInsight**
Interface graphique officielle d'exploration et de diagnostic
-  **redis_exporter + Prometheus/Grafana**
Chaîne de monitoring open-source de référence

Code source & écosystème

-  **github.com/redis/redis**
Code source et release notes
-  **valkey.io**
Fork open-source porté par la Linux Foundation

Pour aller plus loin

-  **Redis University**
Cours
-  **Redis persistence demystified**
Article d'Antirez (2012) sur RDB et AOF